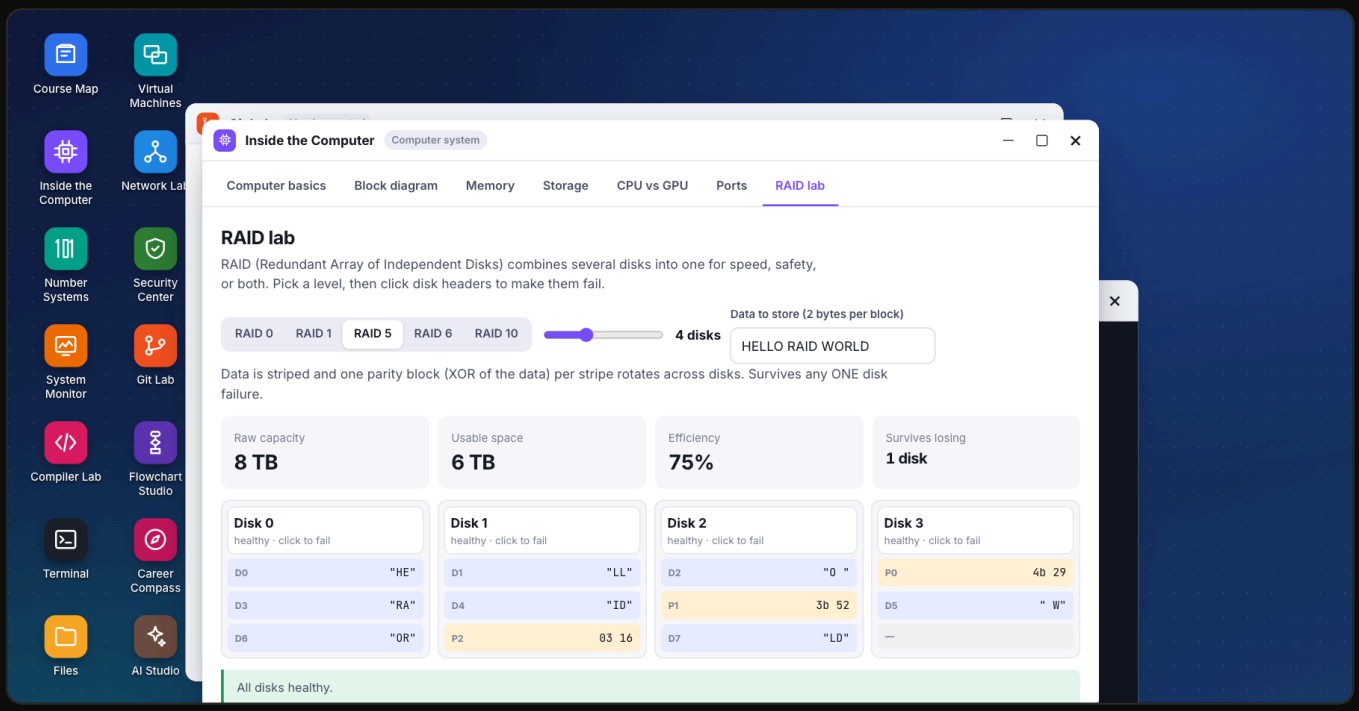
ByteOS

A computer that runs in the browser. It boots like a real PC and every app on its desktop lets you use a part of the syllabus: a Linux terminal with real permissions and git, a RAID lab, a compiler, network and security labs, flowcharts, careers and emerging tech.



STUDENT	Lukshya Singh Supyal	REGISTRATION NO.	
PROGRAMME	B.Tech CSE, Semester 5	COURSE	CSE111 Orientation to Computing-I
LIVE WEBSITE	lushadowx.github.io/ByteOS	SOURCE CODE	github.com/LuShadowX/ByteOS

CONTENTS

What's in this report

01 **Project overview**
Idea, apps, numbers

02 **How it's built**
Architecture and files

03 **Boot, login and desktop**
The OS shell

04 **Unit 1 · Computer system**
Inside the Computer, Number Systems

05 **Unit 2 · Operating system**
System Monitor, Compiler Lab

06 **Unit 3 · Linux**
Terminal, Files, Virtual Machines

07 **Unit 4 · Cohorts and skill sets**
Career Compass

08 **Unit 5 · Networks and security**
Network Lab, Security Center

09 **Unit 6 · Version control, AI, profiles**
Git Lab, AI Studio

10 **Computational thinking**
Flowchart Studio

11 **Syllabus and CO mapping**
Every topic, both syllabus versions

12 **Code walkthrough**
How the important parts work

13 **Testing**
What was checked and the results

14 **Deployment and conclusion**
GitHub Pages, what I learned

> Every screenshot was taken from ByteOS while actually using it. Every code extract is copied straight from the source files, with file names and line numbers.

01 · PROJECT OVERVIEW

Learning computing by using a computer

CSE111 is a theory course, so the usual risk is that topics like kernels, RAID or phishing stay as definitions. ByteOS turns each topic into something you can try. You boot it, log in as a user with limited permissions, open apps from the desktop, and every app is a working model of one part of the syllabus.

14
apps on the desktop

58
interactive screens (tabs)

~7,000
lines of JavaScript, HTML and CSS

6 / 6
units and course outcomes covered

The apps

APP	WHAT YOU DO IN IT	SYLLABUS AREA
Course Map	See every syllabus topic, open the app that teaches it, and track your progress	All units
Inside the Computer	Block diagram with a live fetch–decode–execute cycle, memory hierarchy, SSD vs HDD race, CPU vs GPU, ports, RAID lab	Unit 1
Number Systems	Base conversions with full working, bit board, text as binary, quiz	Current Unit I
System Monitor	Live processes, kernel types, OS types, boot process, CPU scheduling	Unit 2
Compiler Lab	Compile or interpret a small language step by step down to machine code	Unit 2
Terminal	A Linux shell with files, permissions, users, sudo, pipes and git	Units 3 and 6
Files	Directory hierarchy, permission bits, file system types	Units 2 and 3
Virtual Machines	Create and run VMs on a limited host, compare Linux and Windows, distros, cloud models	Unit 3
Network Lab	Break topologies, watch a switch learn MAC addresses, follow a web request	Unit 5
Security Center	Crack-time estimates, rainbow tables, working 2FA codes, phishing, malware, accounts, firewall	Unit 5
Git Lab	Live commit graph of the repository you build in the terminal, guided missions	Unit 6
Flowchart Studio	Run algorithms as pseudocode and flowcharts with a trace table	CO4
Career Compass	Cohorts, pathways, MOOCs, hackathons, profile checker, cohort quiz	Units 4 and 6
AI Studio	AI basics, GenAI tools, prompt analyser, ethics, agentic AI, IoT, blockchain, Web3	Unit 6, CO6

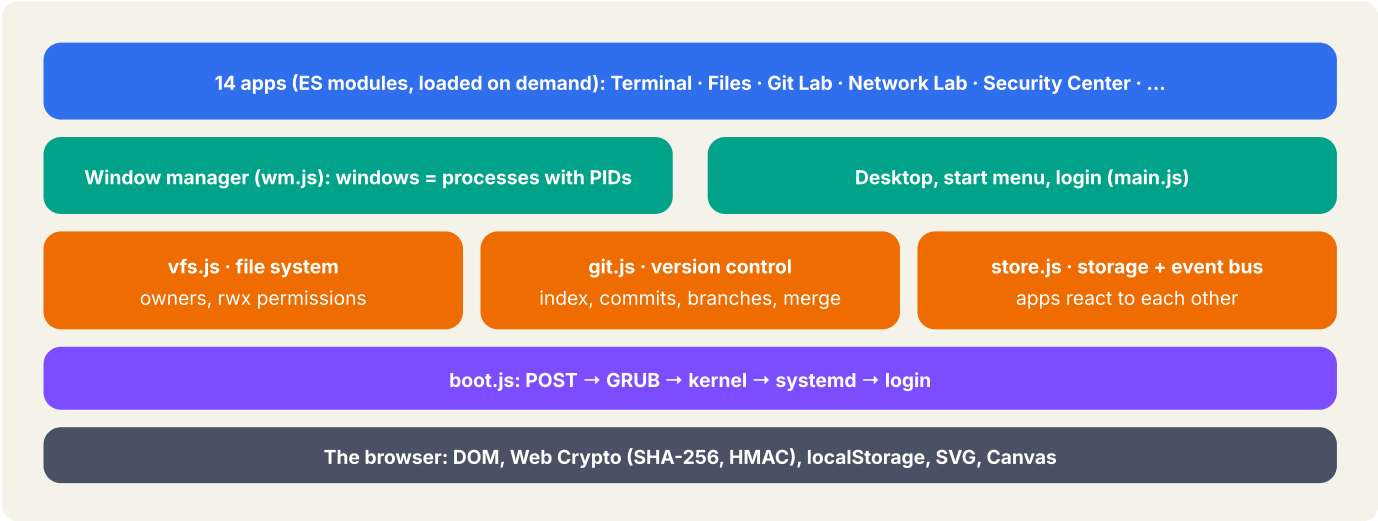
Technology

Plain HTML, CSS and JavaScript (ES modules). There is no framework and no server: everything runs in the browser and saves to the browser's local storage. The site is hosted for free on GitHub Pages.

02 · HOW IT'S BUILT

Architecture

ByteOS copies the structure of a real operating system on purpose. A boot sequence hands over to a login screen. A window manager runs apps as processes. All apps share one file system, and that file system enforces permissions.



The layers mirror the "OS architecture" diagram in the syllabus. Applications sit on top. They talk to shared services (the file system and git) instead of touching storage directly, the same way real apps make system calls instead of touching hardware.

Files

FILE	JOB	LINES
`js/core/boot.js`	The four boot stages with an explanation for each	180
`js/core/wm.js`	Window manager: open, focus, drag, resize, minimise, PIDs	167
`js/core/vfs.js`	Linux-style file system with users, owners and permission checks	275
`js/core/git.js`	Git commands working on the same file system	364
`js/apps/terminal.js`	Shell: parser, pipes, redirection, 45 commands	655
`js/apps/hardware.js`	Computer basics, CPU cycle, memory, storage, GPU, ports, RAID	552
`js/apps/compiler.js`	Lexer, parser, code generator, virtual machine, interpreter	465
`js/apps/security.js`	Password analysis, hashing, TOTP, phishing, firewall	455
`js/apps/network.js`	Topologies with failure, devices, client–server	390
`js/apps/monitor.js`	Processes, kernels, OS types, boot, scheduling	330
`js/apps/*.js` **(others)**	Numbers, Files, VMs, Git Lab, Flowchart, Careers, AI, Course Map	—
`css/base.css` , `css/apps.css`	All styling, responsive down to phone width	—

03 · BOOT, LOGIN AND DESKTOP

It starts like a real computer

Opening the site powers the machine on. Each boot stage shows a short note at the bottom explaining what a real PC is doing at that moment. This covers the bootloader and boot process topics from Unit 2.

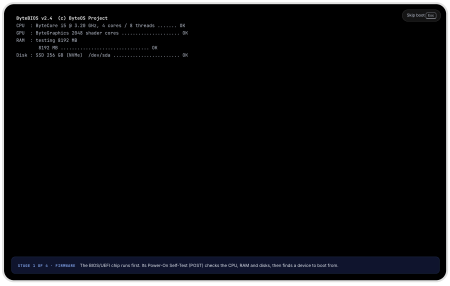


Fig. 1 · Stage 1 · Firmware. The BIOS runs the Power-On Self-Test: CPU, GPU, a RAM count-up and the disk.

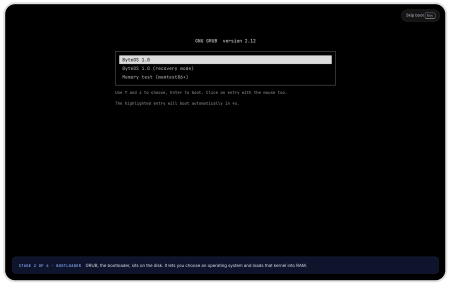


Fig. 2 · Stage 2 · Bootloader. A GRUB menu with recovery mode and a memory test. Arrow keys and Enter work.

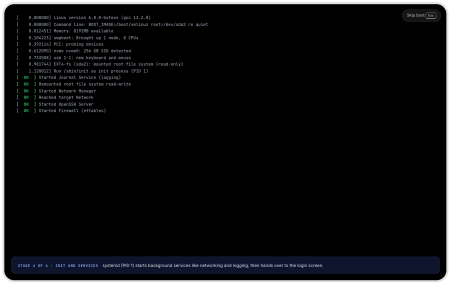


Fig. 3 · Stages 3–4 · Kernel and services. Kernel messages, then systemd starting services with [OK] lines.

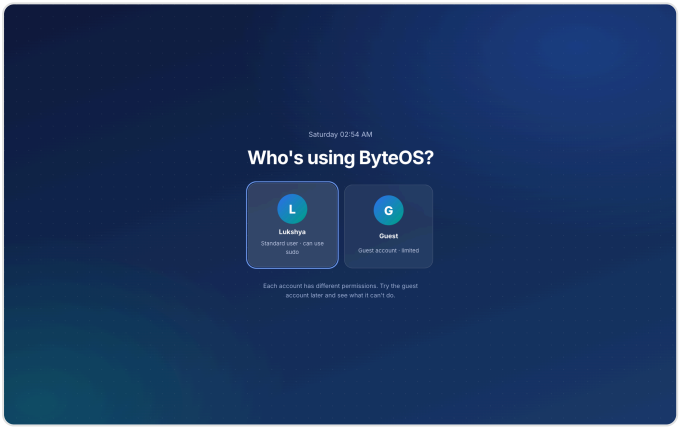


Fig. 4 · Login. Two accounts: a standard user (can use sudo) and a guest. Permissions really differ between them.

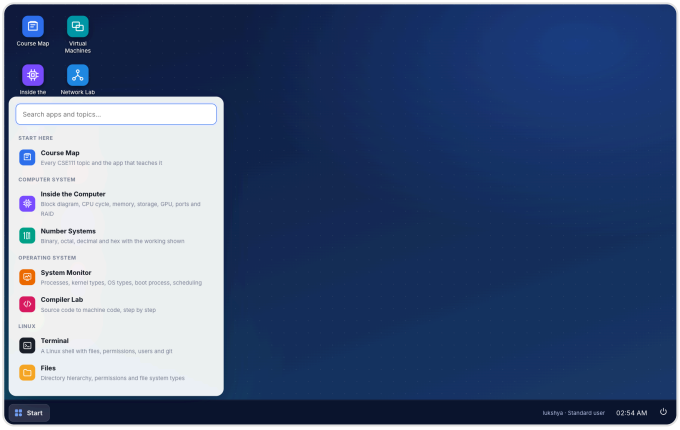


Fig. 5 · Desktop and start menu. Apps grouped by syllabus area, with search.

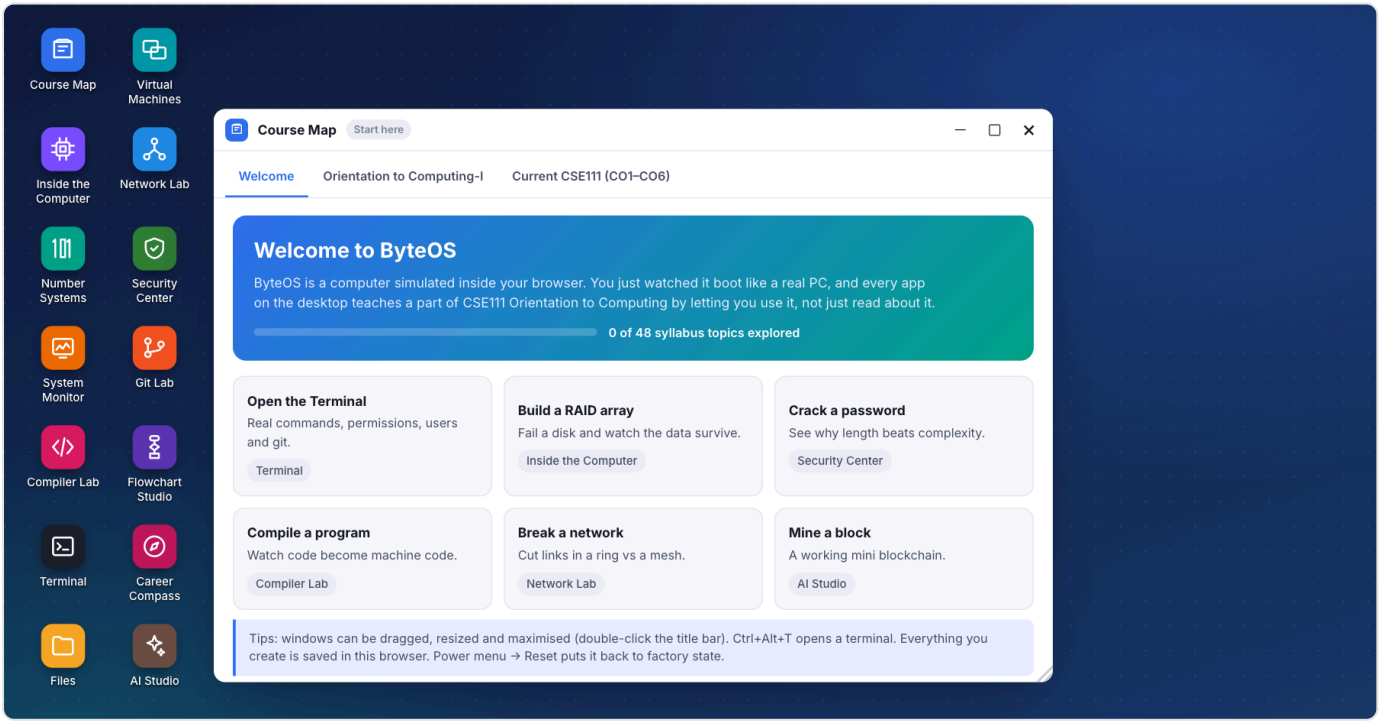


Fig. 6 · Course Map opens on first login. It explains ByteOS and links to quick starts in each app.

Inside the Computer

Seven tabs cover the whole of Unit 1: the structure of a computer, memory, storage, processors, interfaces and RAID. The current syllabus's evolution, characteristics and types of computers are here too.

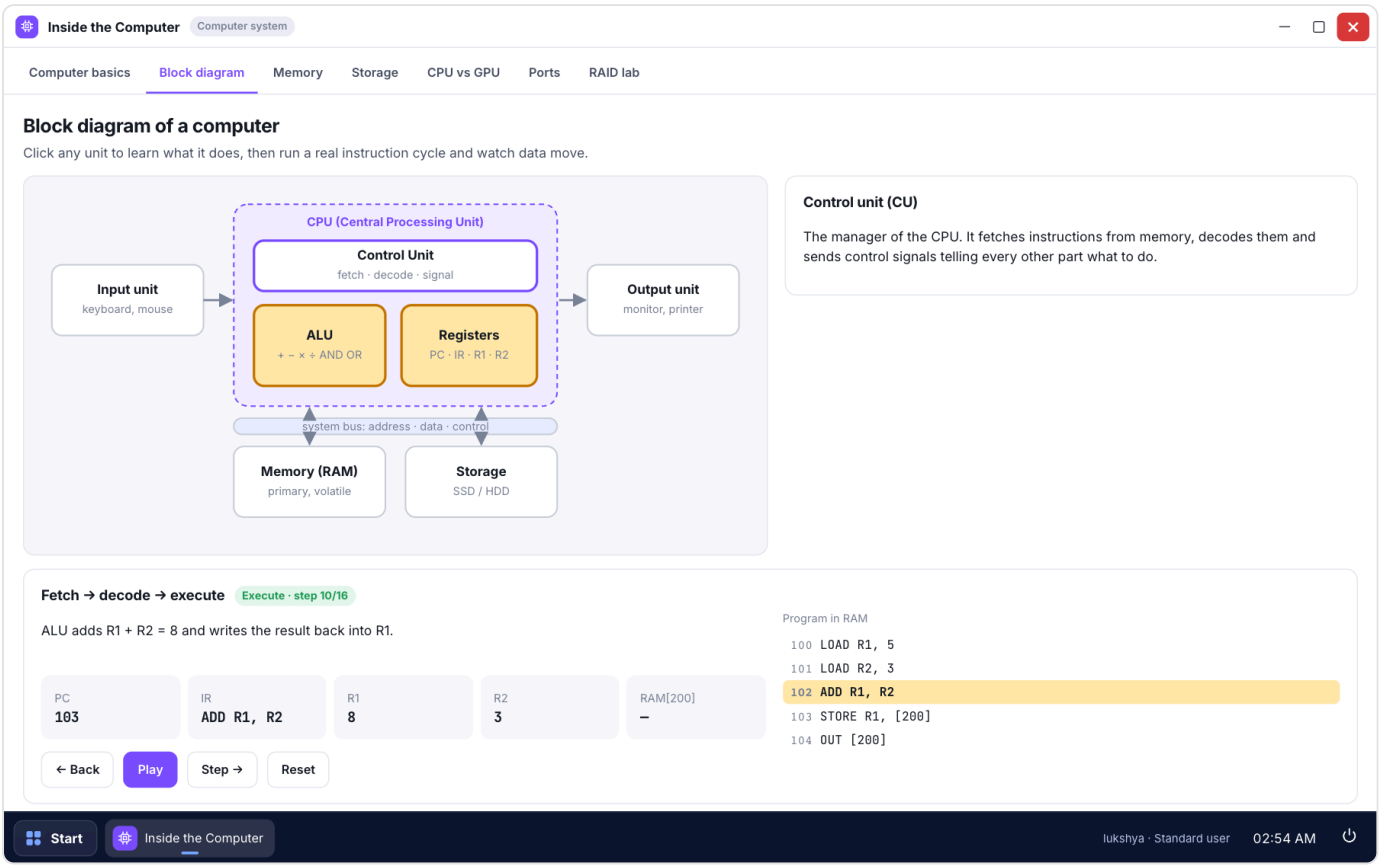


Fig. 7 · **Block diagram + instruction cycle.** Each unit (input, CU, ALU, registers, memory, storage, output, bus) is clickable. "Step" runs a 5-instruction program for A = 5 + 3. The active units light up, and PC, IR, R1, R2 and RAM update at every fetch, decode and execute.

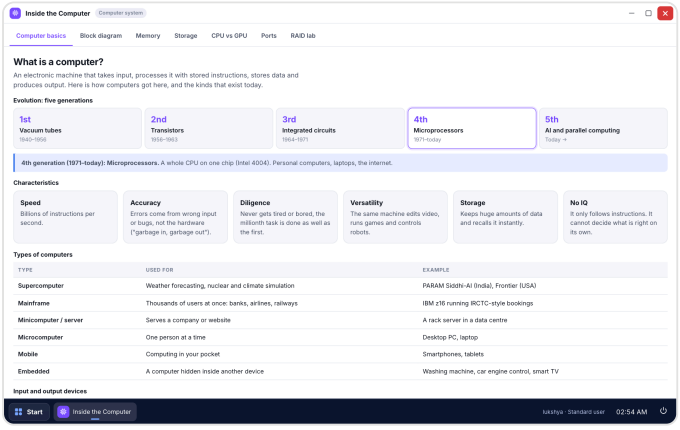


Fig. 8 · **Computer basics.** Five generations, characteristics, types of computers, I/O devices, and a hardware vs software sorting game.

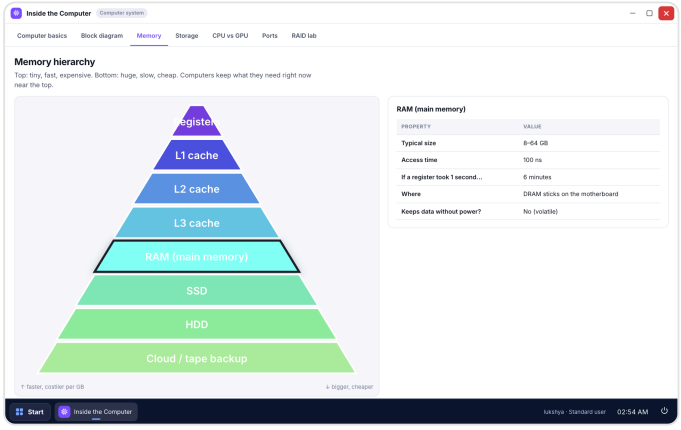


Fig. 9 · **Memory hierarchy.** Click a level for size, speed and volatility, plus "if a register took 1 second" to make the speed gaps feel real. RAM and ROM families below.

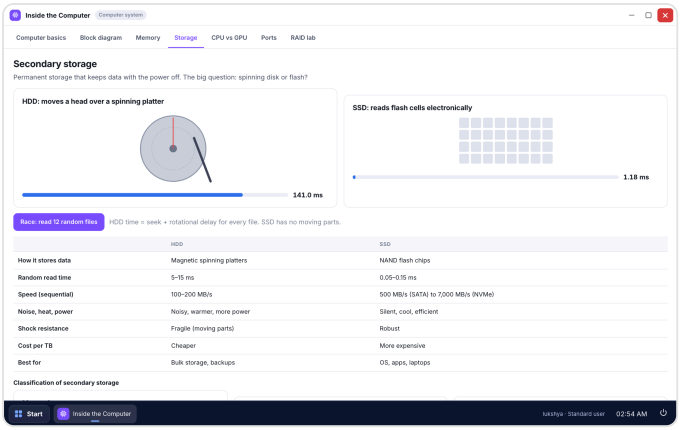


Fig. 10 · **SSD vs HDD race.** 12 random reads: the HDD pays seek and rotation time for each (150 ms), the SSD about 1 ms.

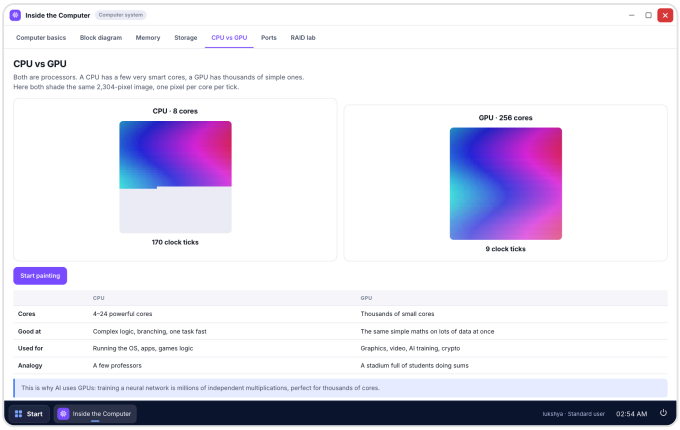


Fig. 11 · **CPU vs GPU.** 8 cores vs 256 cores paint the same image, one pixel per core per tick: 169 ticks vs 9.

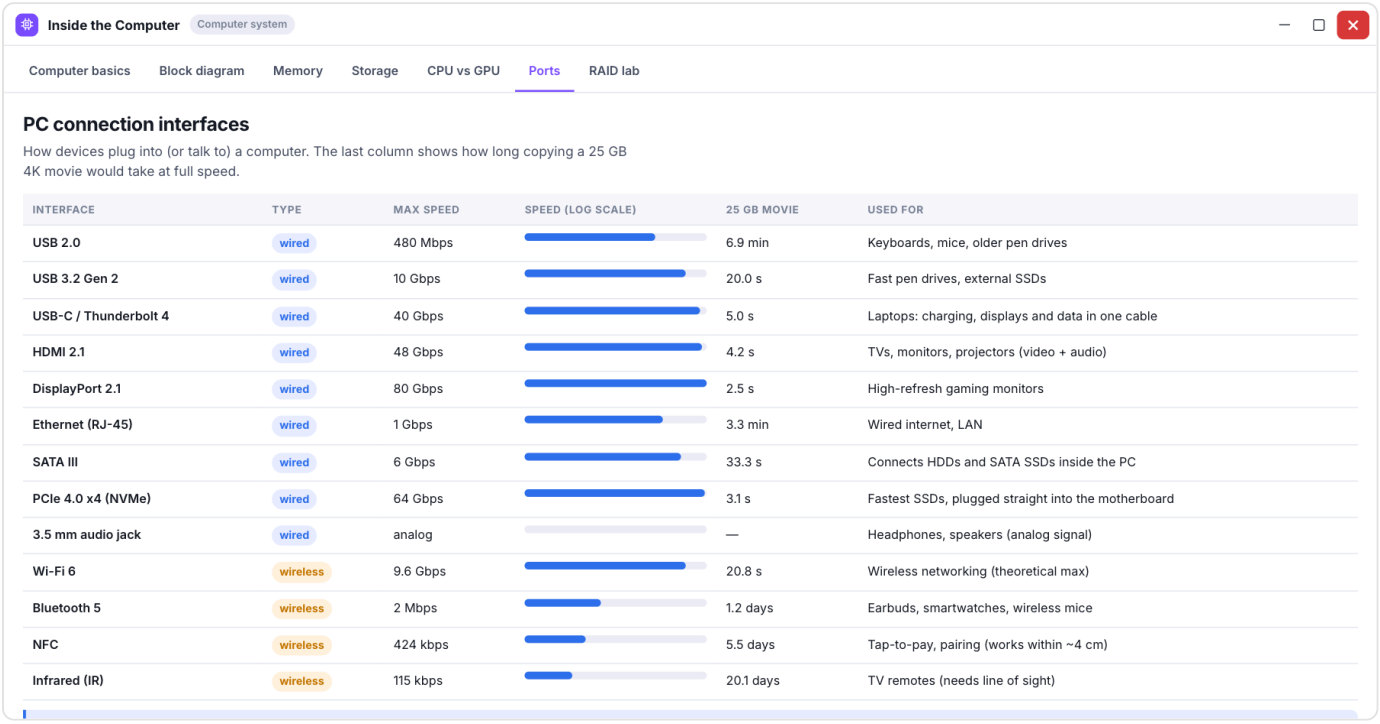


Fig. 12 · **PC connection interfaces.** Wired and wireless ports with maximum speeds on a log scale and the time to copy a 25 GB file over each.

RAID lab

Choose RAID 0, 1, 5, 6 or 10 and 2–6 disks. Your text is split into blocks and laid out exactly as that level would: striped, mirrored, or striped with rotating parity. The parity is computed for real: P is the XOR of the data blocks, and Q uses Galois-field multiplication. Click a disk to fail it.

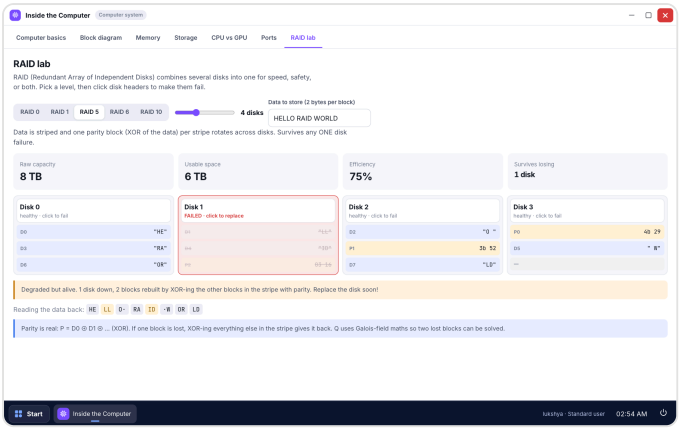


Fig. 13 · **RAID 5, one disk failed:** "degraded but alive". The missing blocks are rebuilt by XOR (shown in yellow) and the data reads back complete.

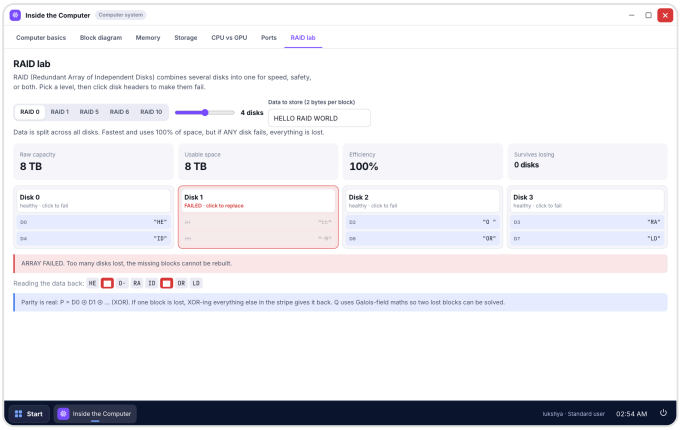


Fig. 14 · **RAID 0, one disk failed:** "array failed". Striping has no redundancy, so the blocks on that disk are lost (red).

Number Systems

Conversions between binary, octal, decimal and hexadecimal with the working written out the way it is done in an exam.

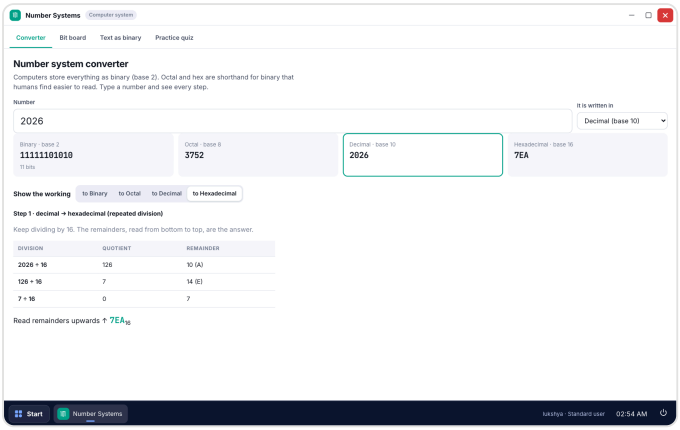


Fig. 15 · Decimal → hex by repeated division. 2026 ÷ 16 step by step. The remainders read upwards give 7EA.

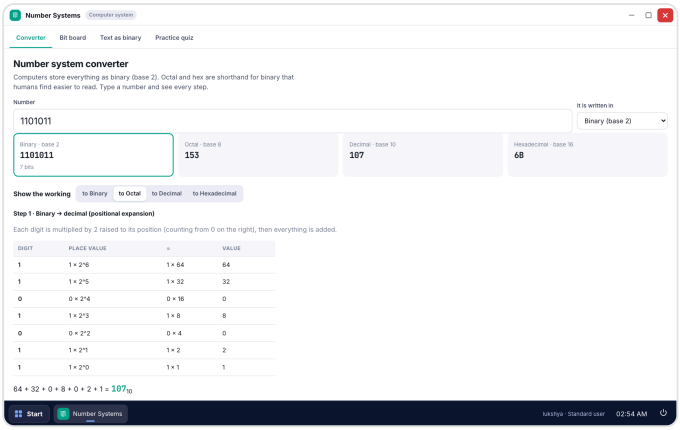


Fig. 16 · Binary → decimal by positional expansion. Each digit × 2^n, summed to 107. The grouping shortcut (3 bits per octal digit) is also shown.

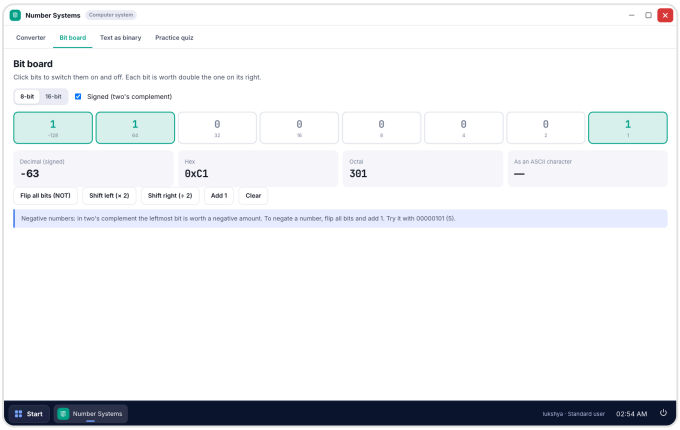


Fig. 17 · Bit board. Toggle bits and see the value in every base. Signed mode shows two's complement: 11000001 = −63.

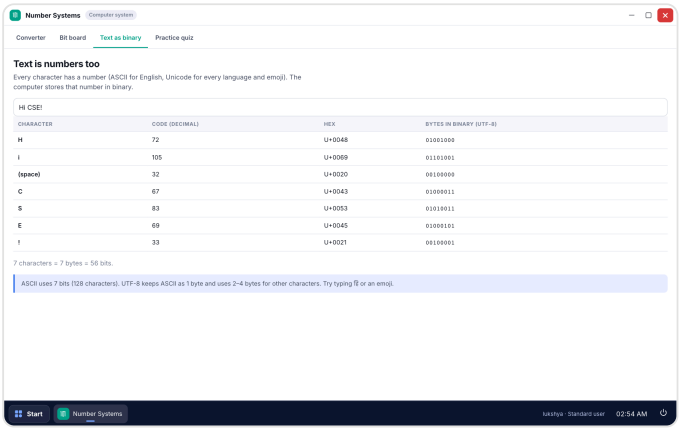


Fig. 18 · Text as binary. Each character's code point and UTF-8 bytes.

System Monitor

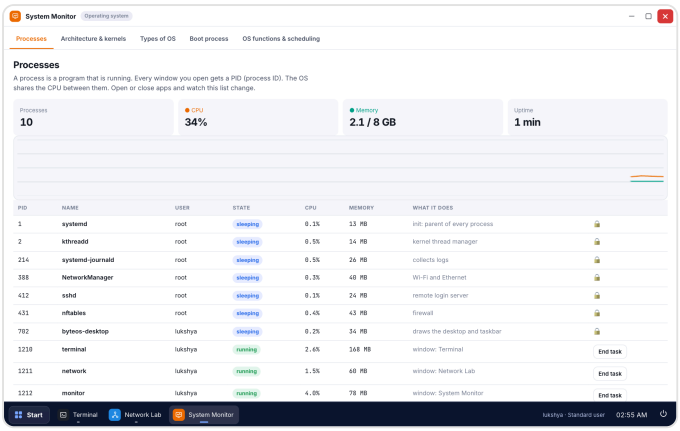


Fig. 19 · **Processes.** System processes (systemd PID 1, sshd, the firewall) plus every open window as a process with a PID. "End task" really closes the window.

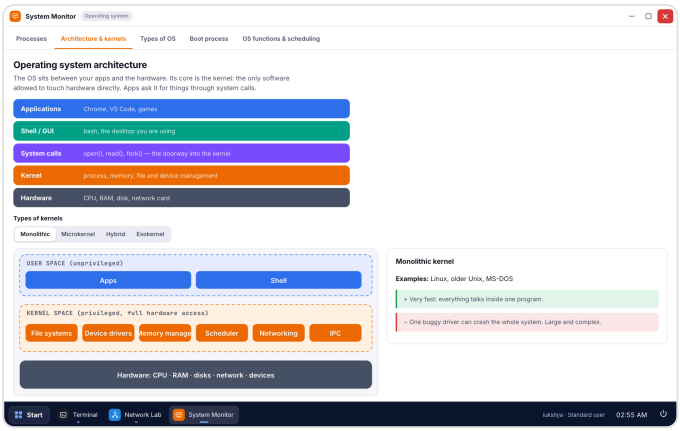


Fig. 20 · **OS architecture and kernel types.** The layers from apps to hardware, and diagrams of monolithic, micro, hybrid and exokernels with examples, pros and cons.

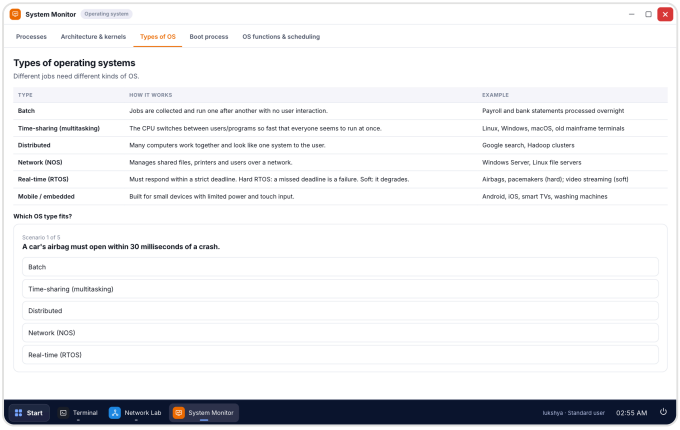


Fig. 21 · **Types of OS.** Batch, time-sharing, distributed, network, real-time and mobile, plus a "which OS fits this scenario?" quiz.

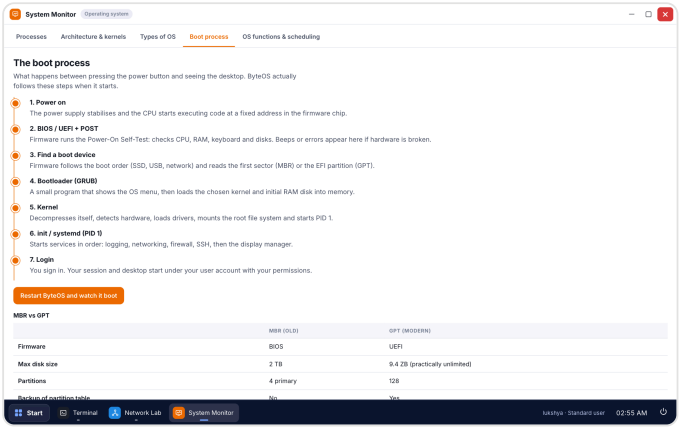


Fig. 22 · **Boot process.** Seven steps from power-on to login, MBR vs GPT, and a button that reboots ByteOS to watch it happen.

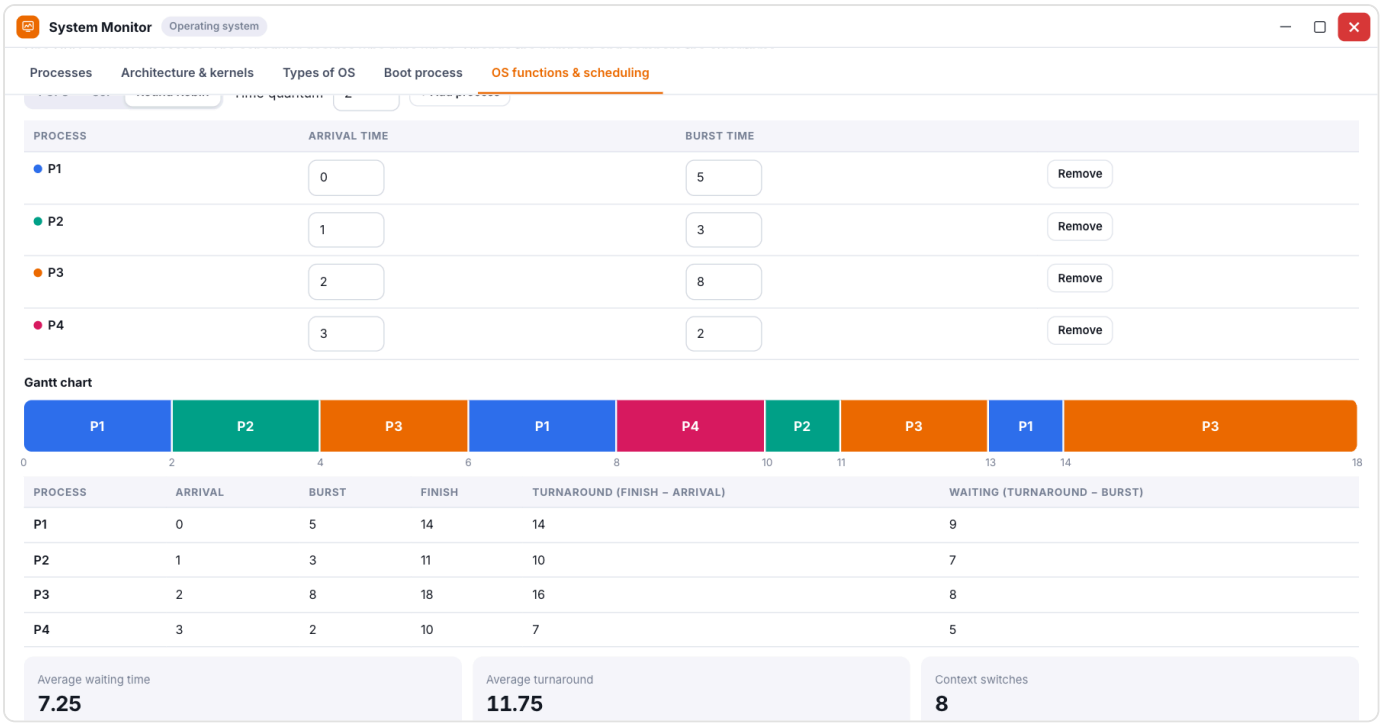


Fig. 23 · **OS functions and CPU scheduling.** Eight OS functions, then a working scheduler: edit arrival and burst times and compare FCFS, SJF and Round Robin. The Gantt chart and the average waiting and turnaround times are calculated live.

Compiler Lab

Covers computer languages, language translators, the program development cycle, and compilation and execution. ByteScript is a tiny language (`let`, `print`, arithmetic, brackets), and ByteOS really compiles it.

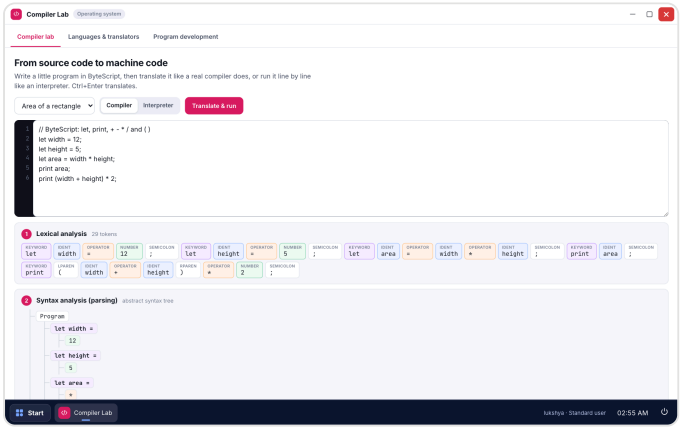


Fig. 24 · **Stages 1–2.** Lexical analysis splits the code into colour-coded tokens. Parsing builds an abstract syntax tree.

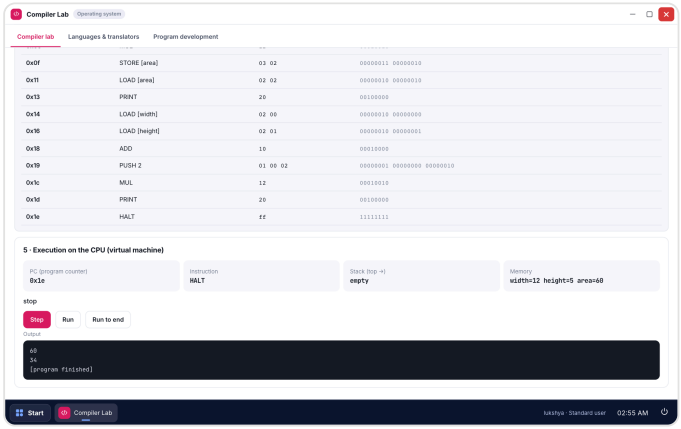


Fig. 25 · **Stages 4–5.** Each assembly instruction with its address and bytes in hex and binary, then a stack VM runs it. Output: 60 and 34.

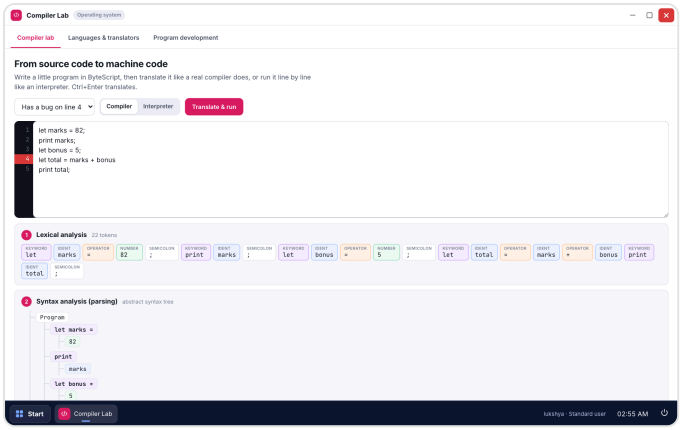


Fig. 26 · **Compiler mode, missing ";" on line 4.** The compiler reports the error at line 4 and runs nothing.

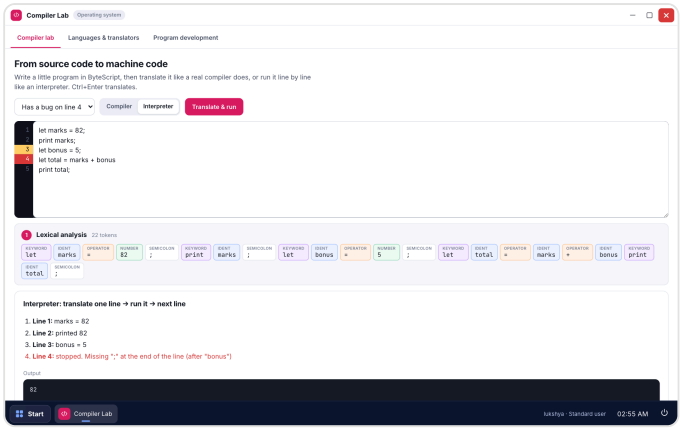


Fig. 27 · **Interpreter mode, same program.** Lines 1–3 are translated and run (82 is printed), then it stops at line 4. This is the compiler vs interpreter difference.

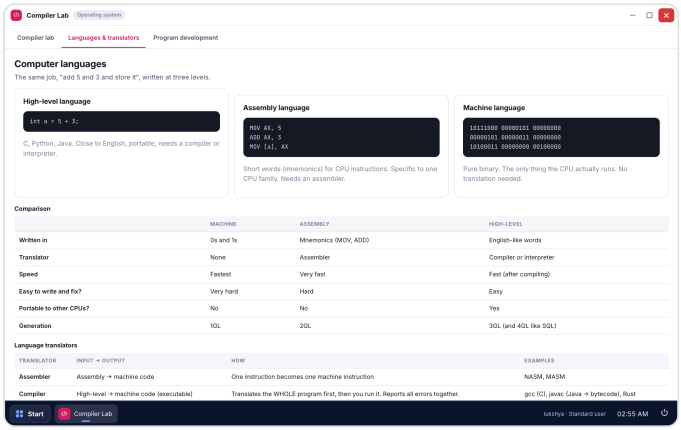


Fig. 28 · **Languages and translators.** One task in high-level, assembly and machine code, and assembler vs compiler vs interpreter.

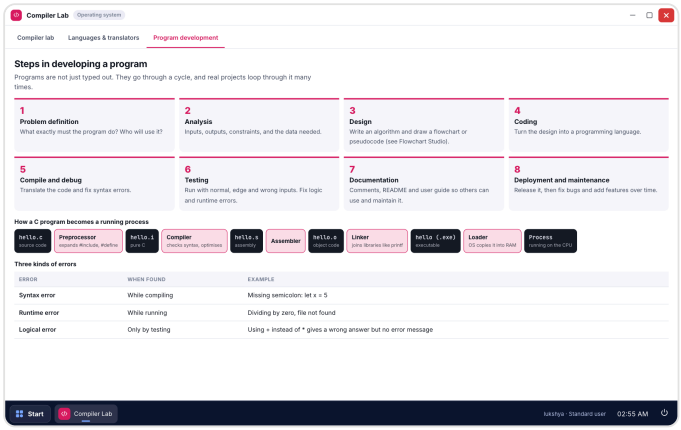


Fig. 29 · **Program development cycle,** the C toolchain (preprocessor → compiler → assembler → linker → loader) and the three kinds of errors.

Terminal

A bash-style shell with 45 commands over a Linux file system (/bin, /boot, /dev, /etc, /home, /root, /tmp, /usr, /var, /proc). It supports quotes, $VARIABLES, * globs, pipes `|`, redirection `> >>`, `&&` chains, Tab completion, history and `man` pages.

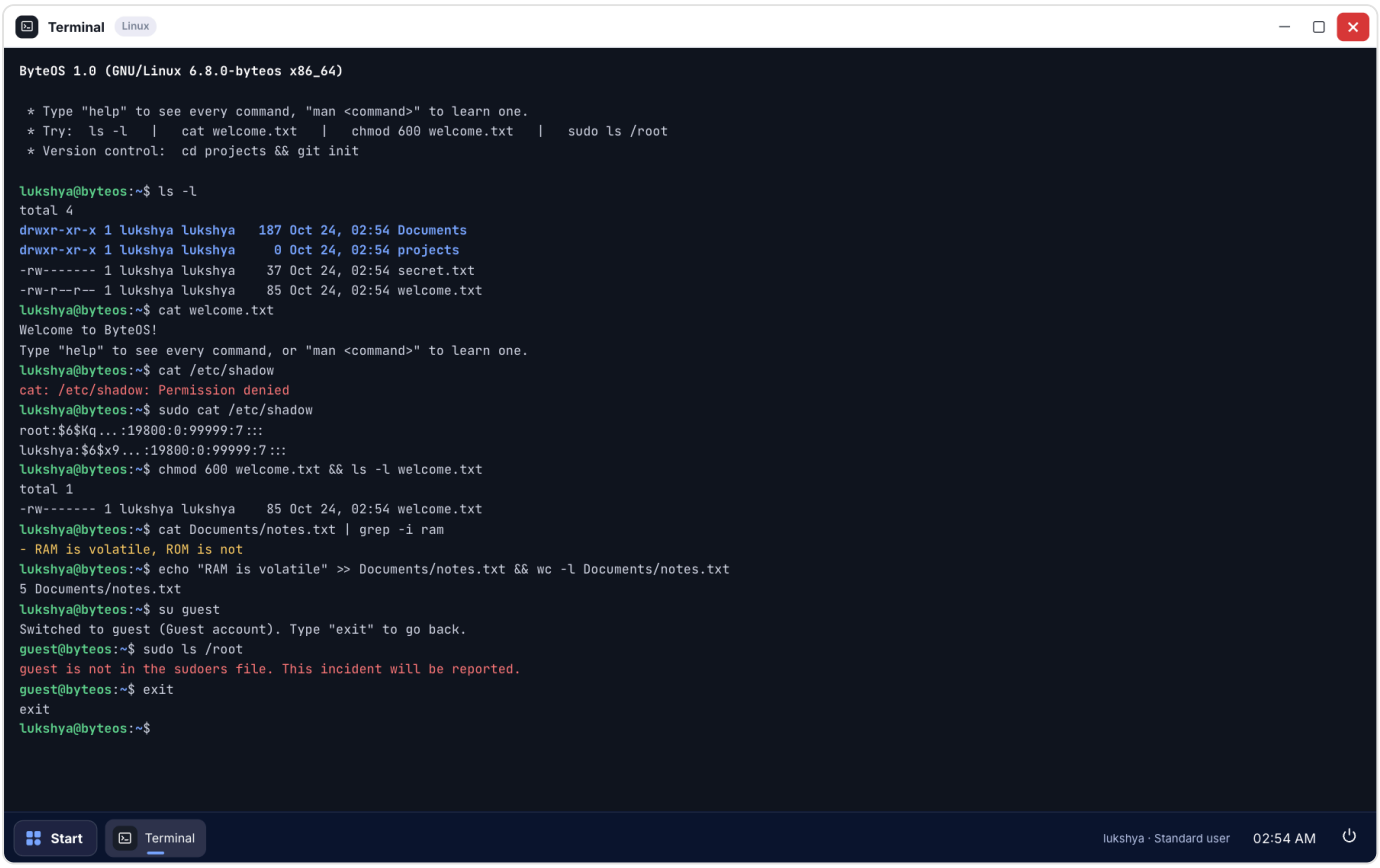


Fig. 30 · **Permissions and users.** `cat /etc/shadow` is denied for a normal user and works with `sudo`. `chmod 600` changes the mode to `-rw-------`. A pipe filters notes with grep, `>>` appends to a file. After `su guest`, `sudo` is refused: "guest is not in the sudoers file".

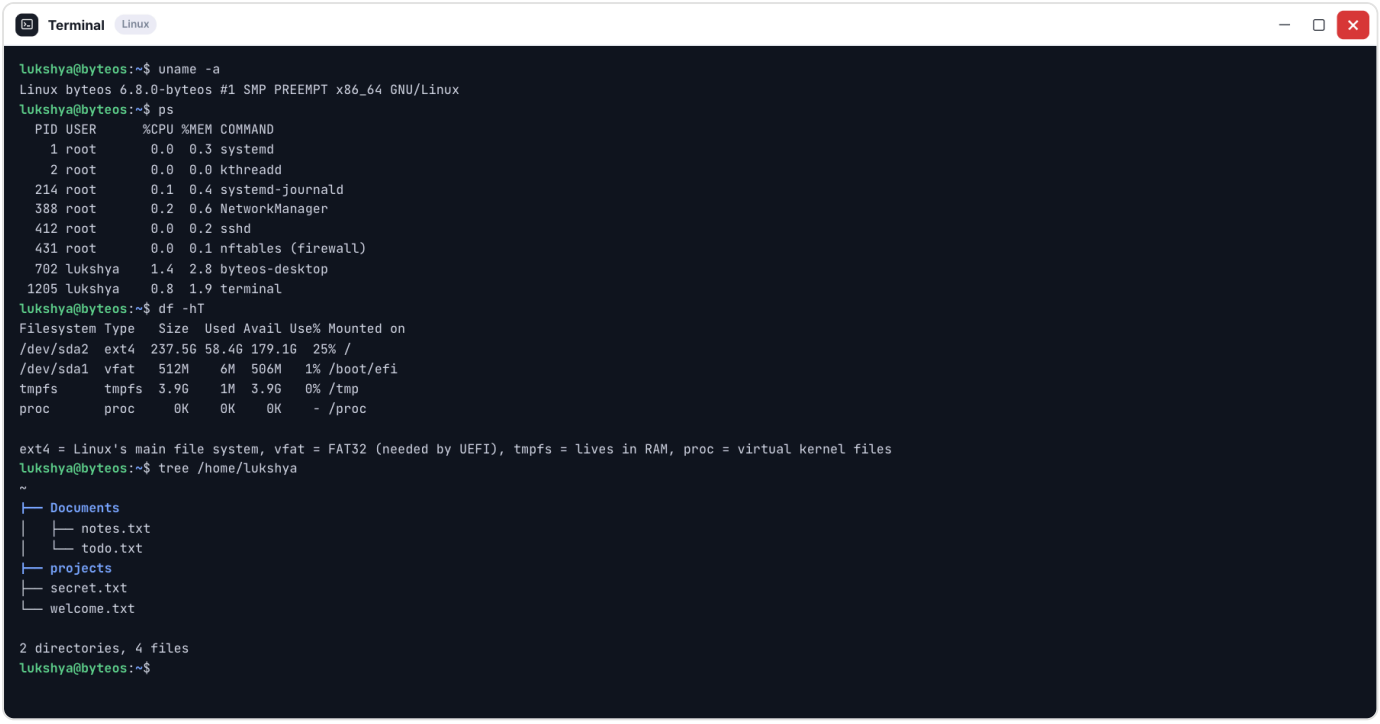


Fig. 31 · **System commands.** `uname -a`, `ps` (window PIDs appear next to system processes), `df -hT` with file system types (ext4, vfat, tmpfs, proc) and `tree`.

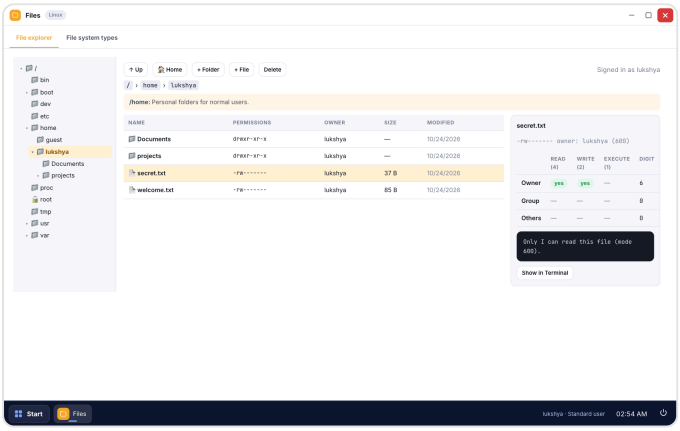


Fig. 32 · **Files.** Directory tree, a description of each top-level folder, and a permission table for the selected file (secret.txt is 600: only the owner can read it).

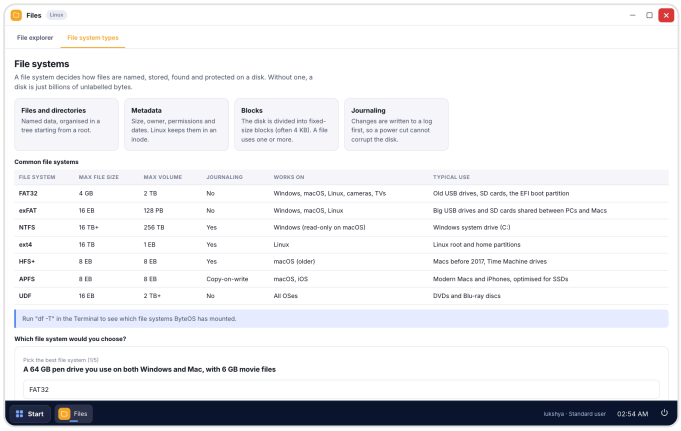


Fig. 33 · **File system types.** FAT32, exFAT, NTFS, ext4, HFS+, APFS and UDF compared, and a "pick the right file system" quiz.

Virtual Machines

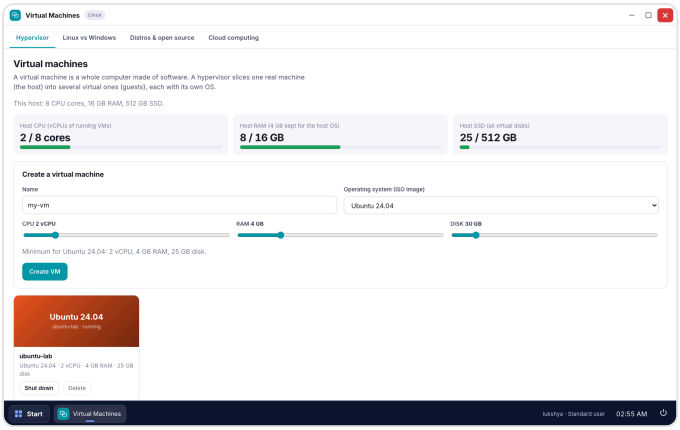


Fig. 34 · **Hypervisor.** Create VMs from OS images with vCPU, RAM and disk sliders. The host's limits are enforced and minimum requirements are checked. Type 1 vs type 2 diagrams and VirtualBox steps follow.

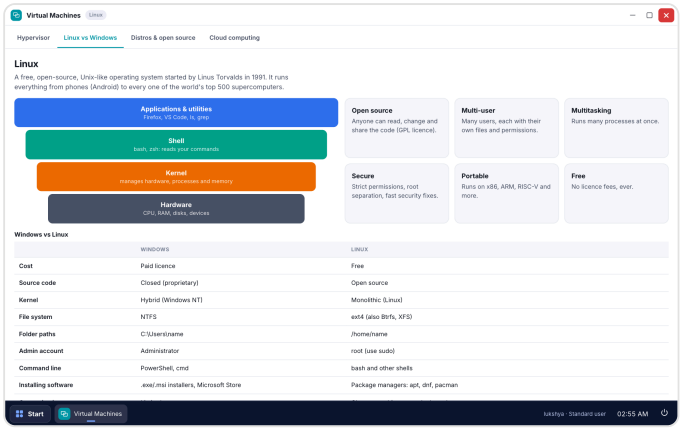


Fig. 35 · **Linux.** Architecture (hardware → kernel → shell → applications), features, and a full Windows vs Linux comparison.

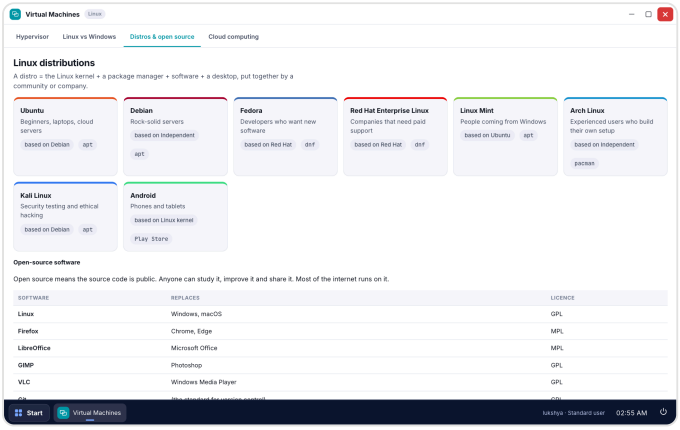


Fig. 36 · **Distributions and open source.** Eight distros with base and package manager, and open-source software with licences.

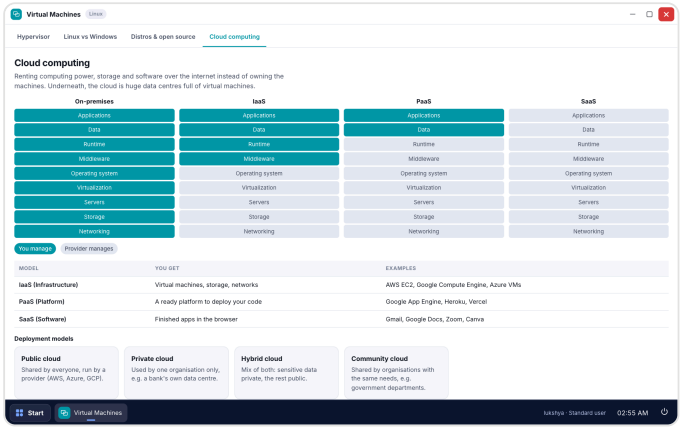


Fig. 37 · **Cloud computing.** Who manages what in on-premises, IaaS, PaaS and SaaS, with examples and deployment models.

Career Compass

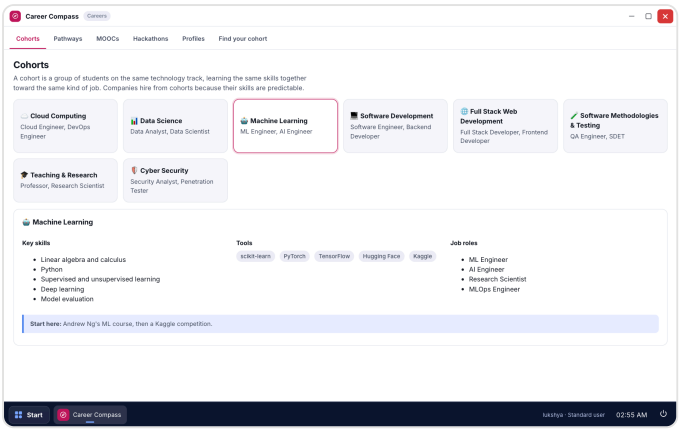


Fig. 38 · **Cohorts.** All eight from the syllabus (Cloud, Data Science, ML, Software Development, Full Stack, Methodologies & Testing, Teaching & Research, Cyber Security) with skills, tools, roles and a first step.

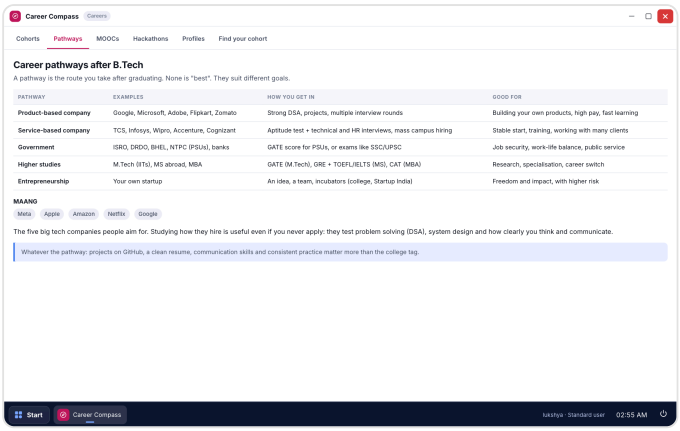


Fig. 39 · **Pathways.** Product-based, service-based, government, higher studies and entrepreneurship, plus MAANG.

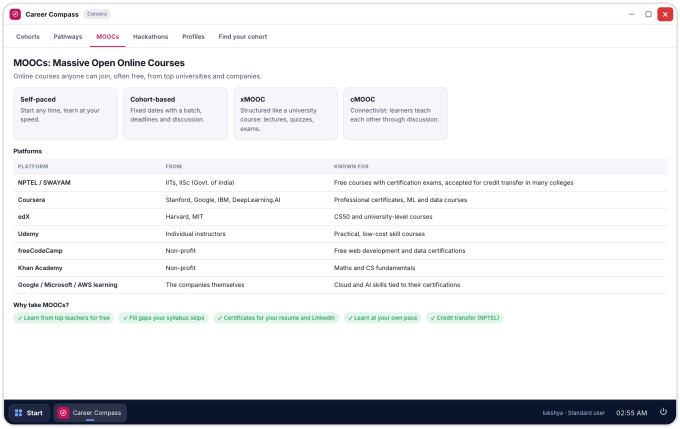


Fig. 40 · **MOOCs.** Types, platforms (NPTEL/SWAYAM, Coursera, edX…) and benefits.

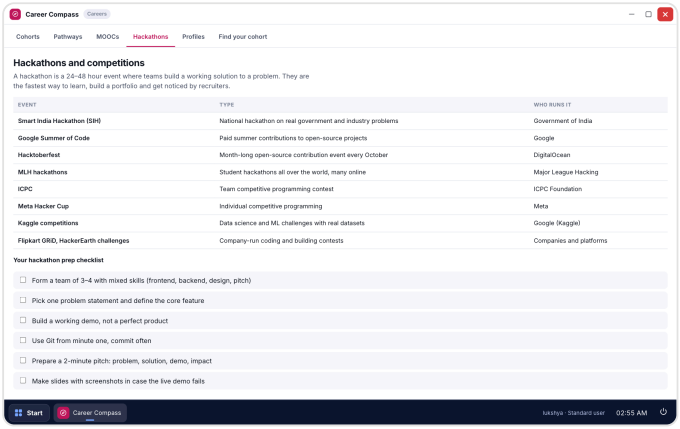


Fig. 41 · **Hackathons.** SIH, GSoC, Hacktoberfest, ICPC, Kaggle and others, with a saved prep checklist.

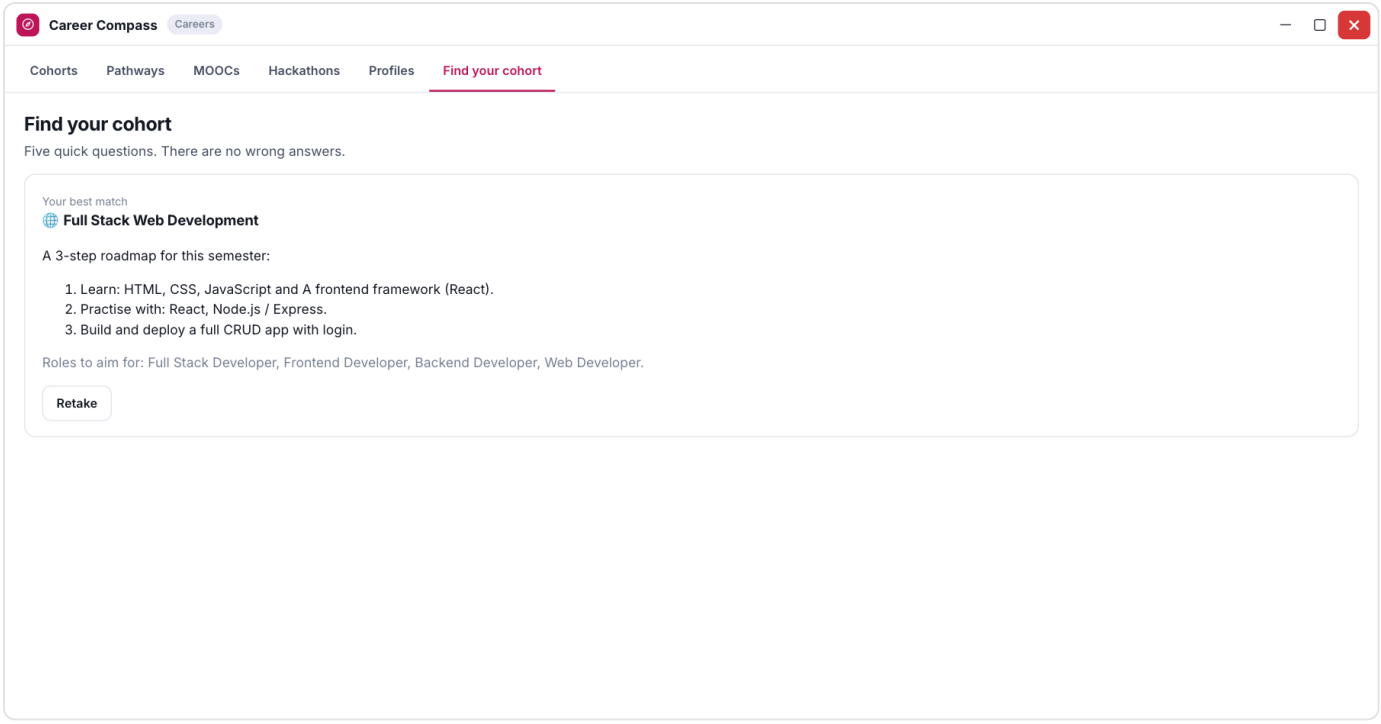


Fig. 42 · **Find your cohort.** Five questions score each cohort and suggest a three-step roadmap.

Network Lab

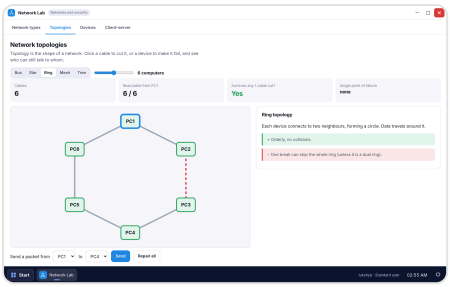


Fig. 43 · **Ring, one cable cut:** still 6/6 reachable, because data goes the other way round.

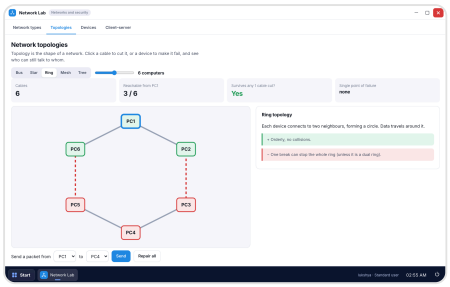


Fig. 44 · **Ring, two cuts:** the network splits, so only 3/6 are reachable.

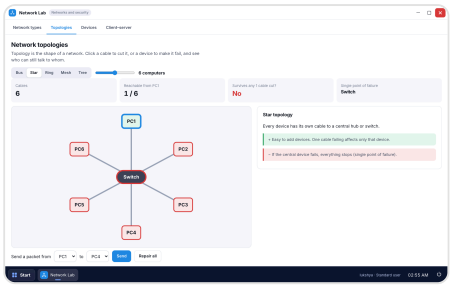


Fig. 45 · **Star, switch failed:** 1/6. The centre is a single point of failure.

Bus, star, ring, mesh and tree are generated for 4–8 computers. Reachability is worked out with a breadth-first search, and the app also tests whether the topology survives any single cable cut. "Send" animates a packet along the route.

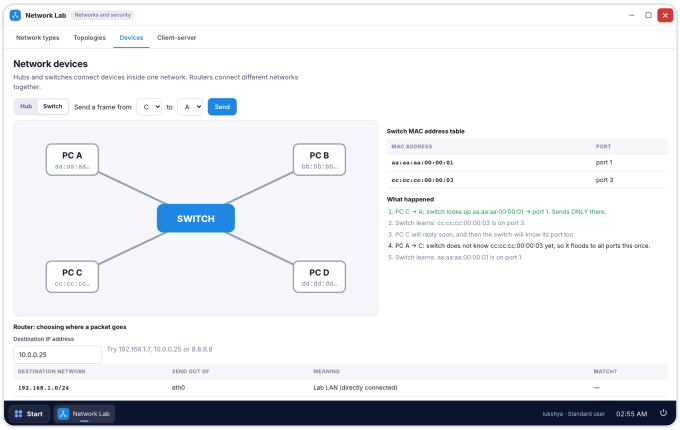


Fig. 46 · **Hub vs switch vs router.** A hub floods every port. A switch learns MAC addresses (table shown) and then sends to one port. Below, a router picks a route from a routing table for any IP you type.

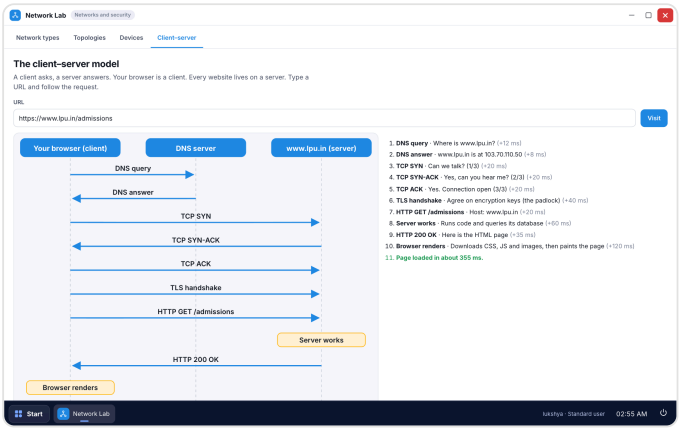


Fig. 47 · **Client–server.** A sequence diagram of loading a URL: DNS, TCP handshake, TLS, HTTP request, server work, response, rendering.

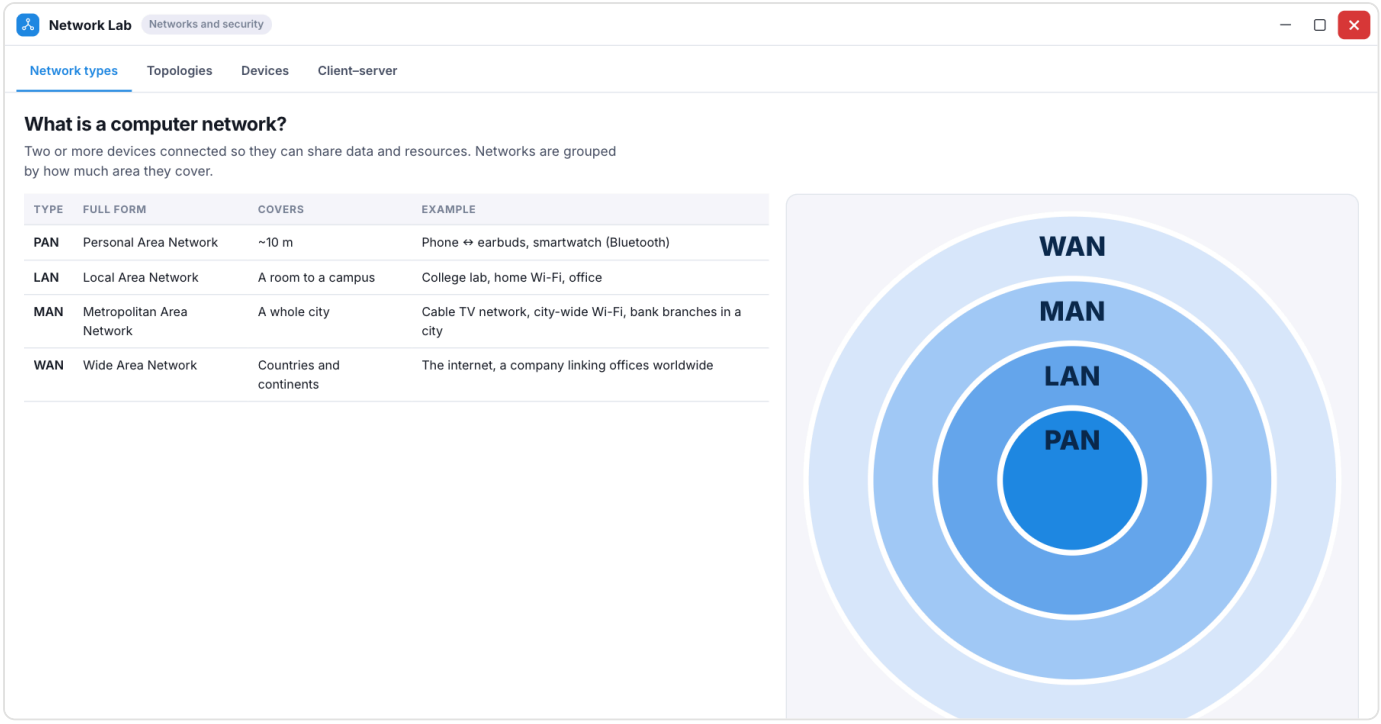


Fig. 48 · **Network types** PAN, LAN, MAN and WAN, the five components of data communication, and transmission media.

Security Center

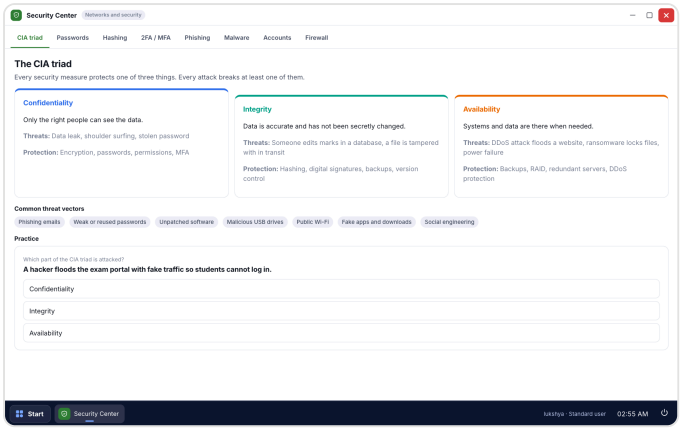


Fig. 49 · **CIA triad,** threats and protection for each, and a "which one is attacked?" quiz.

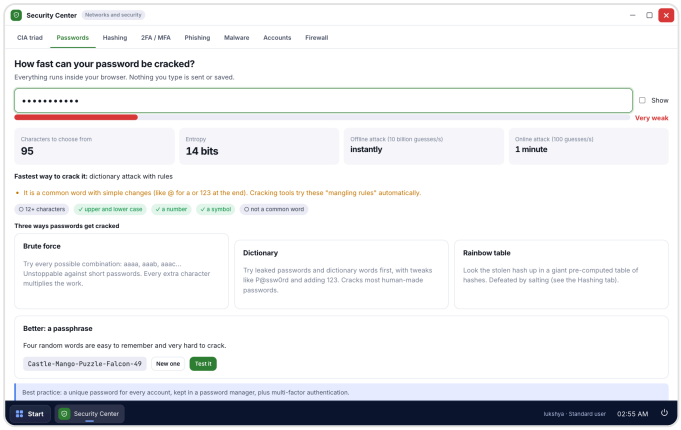


Fig. 50 · **Password cracking.** "P@ssw0rd123" is caught as a dictionary word with tweaks (14 bits). Crack time for brute force offline vs online, and a passphrase generator.

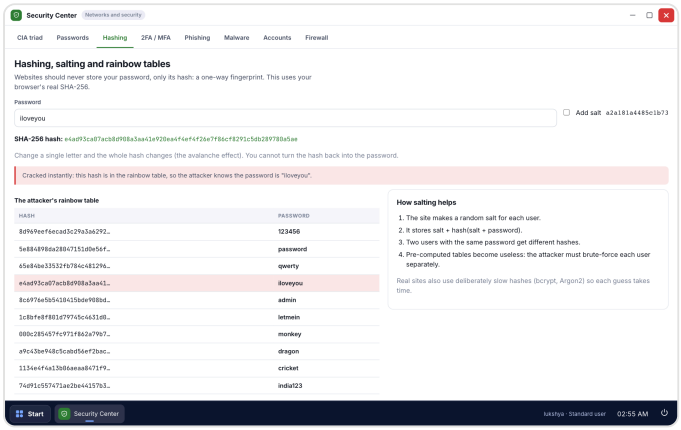


Fig. 51 · **Hashing and rainbow tables.** Real SHA-256. The unsalted hash of "iloveyou" is found in the attacker's table instantly. With a salt it is not.

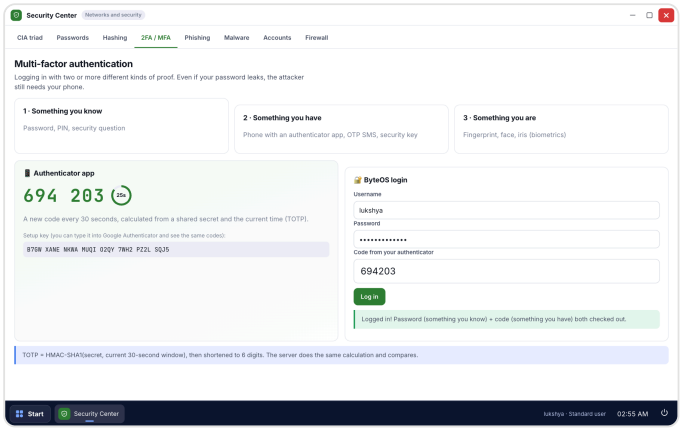


Fig. 52 · **Working two-factor authentication.** The authenticator shows a 6-digit TOTP code every 30 s. Login needs password + code. The setup key also works in Google Authenticator.

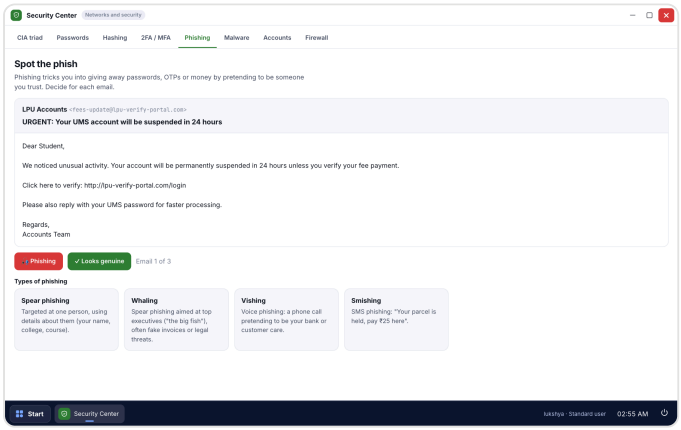


Fig. 53 · **Phishing.** Judge real-looking emails, then the red flags are highlighted. Spear phishing, whaling, vishing and smishing are explained.

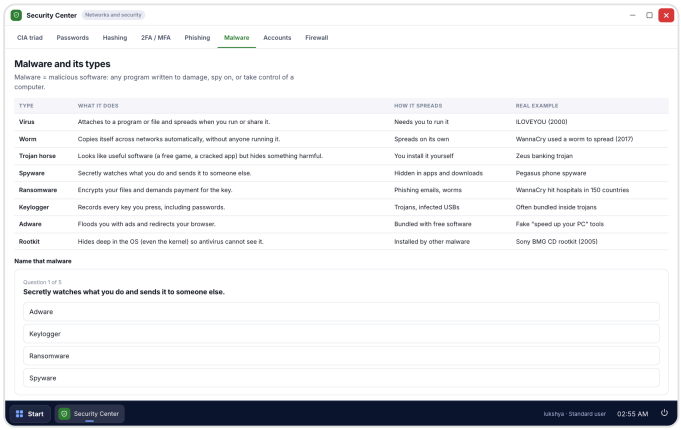


Fig. 54 · **Malware.** Virus, worm, trojan, spyware, ransomware, keylogger, adware and rootkit with real examples and a quiz.

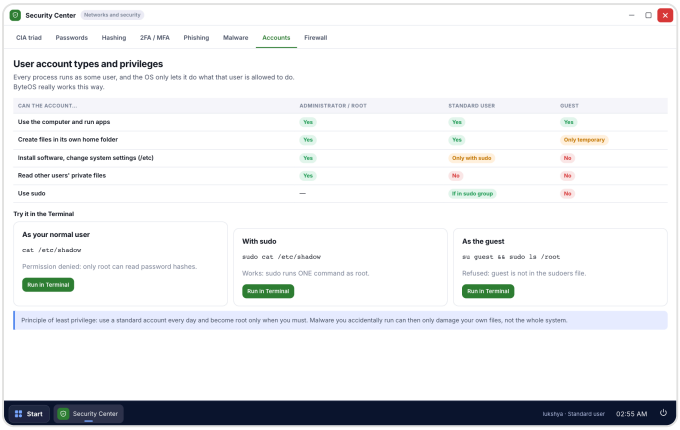


Fig. 55 · **User account types** (root, standard, guest), with buttons that run the matching commands in the Terminal.

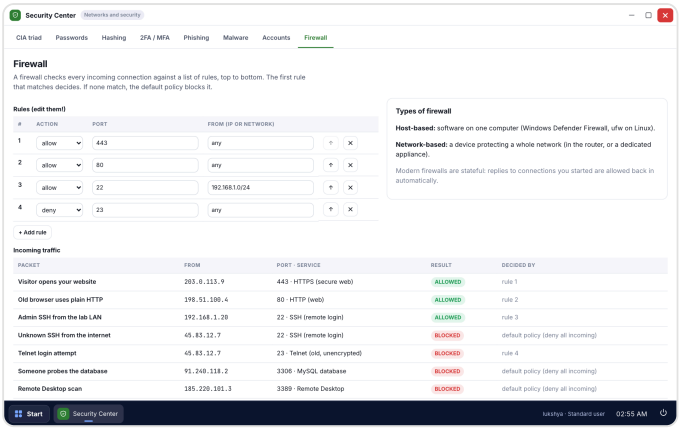


Fig. 56 · **Firewall.** Editable rules checked top to bottom, first match wins, default deny. Seven test packets show allowed or blocked and which rule decided.

Git in the Terminal + Git Lab

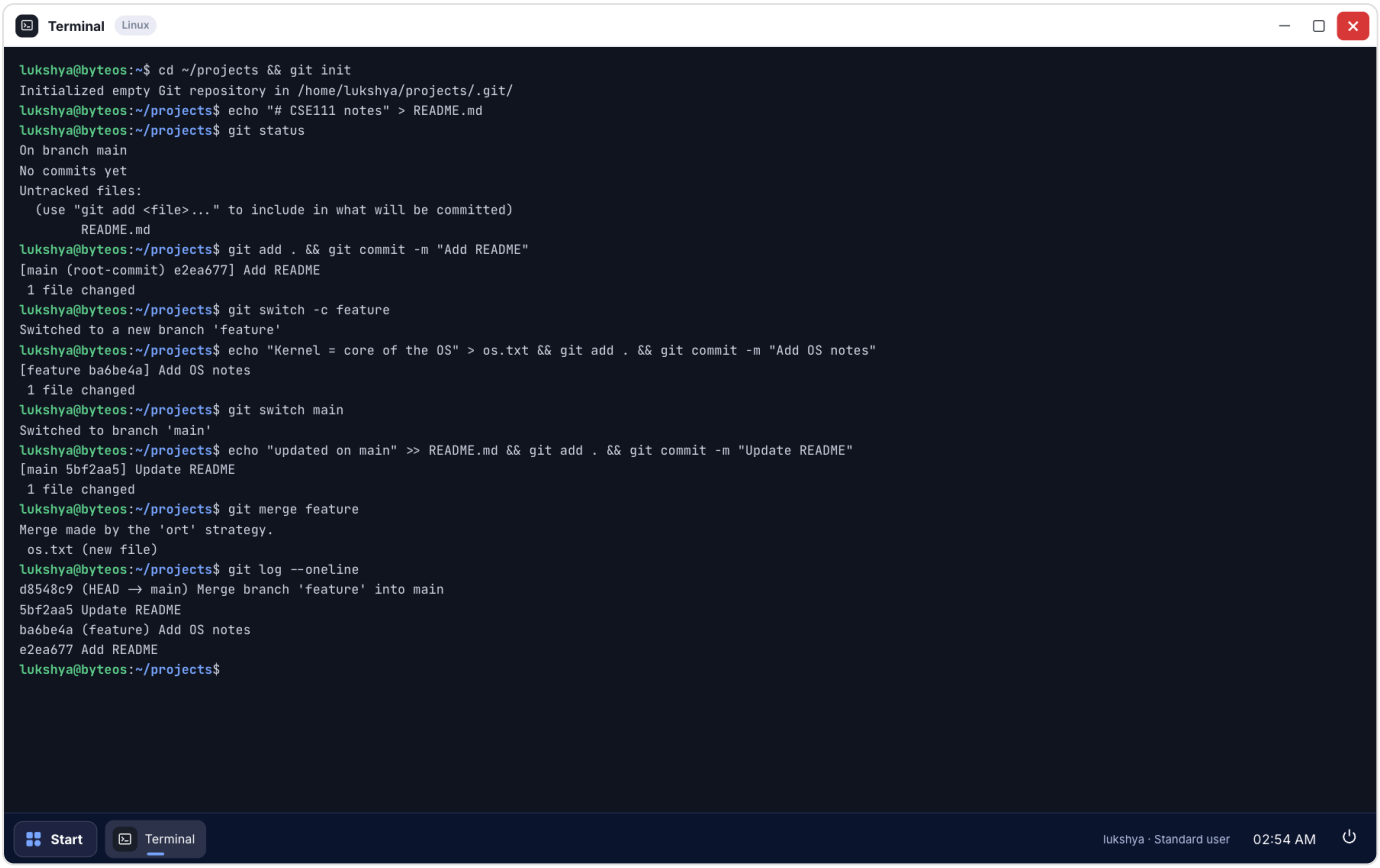


Fig. 57 · **A complete git workflow** typed in the Terminal: init, add, commit, create and switch to a branch, commit there, switch back, commit on main, merge (a real three-way merge), and log.

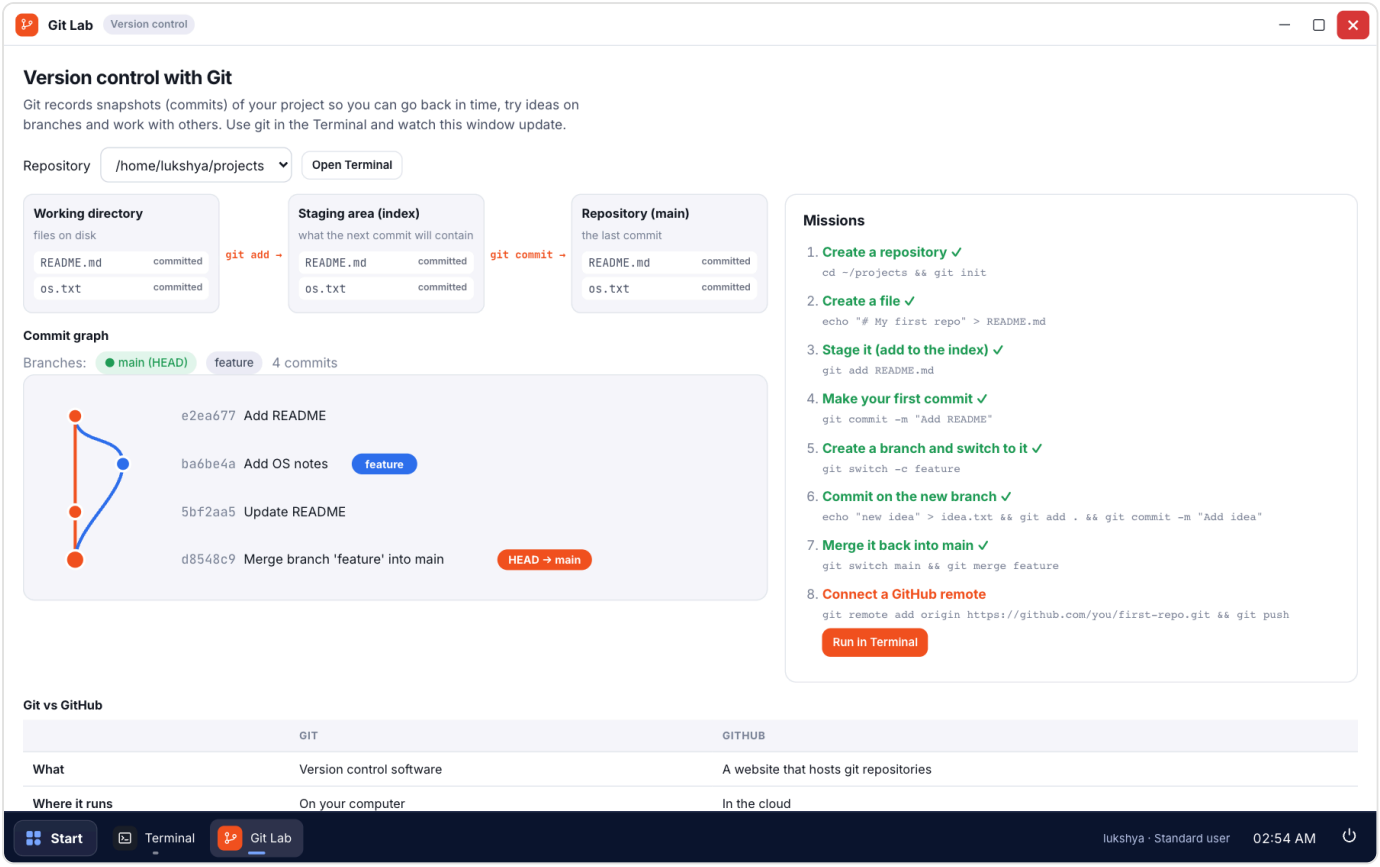


Fig. 58 · **Git Lab** draws the same repository live: working directory → staging area → repository, the commit graph with the feature branch merging back, and missions that tick off automatically (7 of 8 done here).

AI Studio

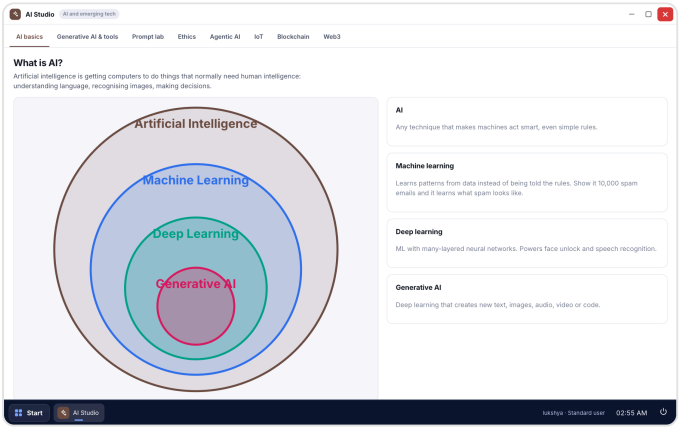


Fig. 59 · **AI basics.** AI ⊃ machine learning ⊃ deep learning ⊃ generative AI, and real-life applications.

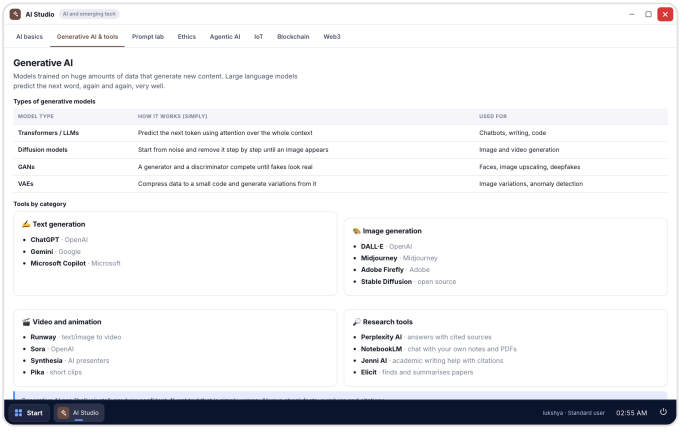


Fig. 60 · **Generative AI and tools.** Model types, and text, image, video and research tools (ChatGPT, Gemini, DALL·E, Midjourney, Runway, Sora, Perplexity AI, NotebookLM, Jenni AI).

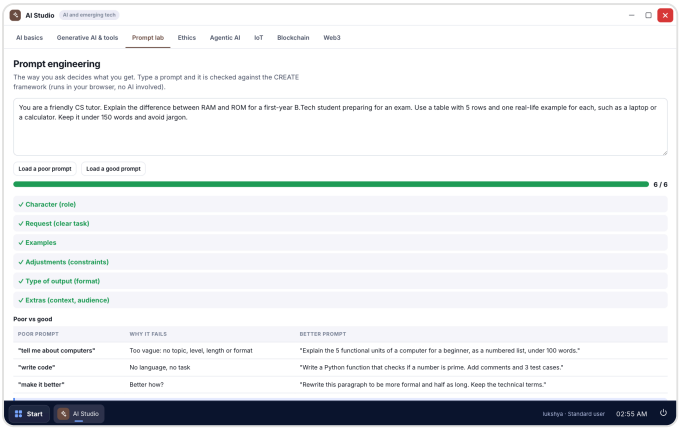


Fig. 61 · **Prompt engineering.** A prompt is scored against the CREATE framework. The good example scores 6/6, the poor one 0/6.

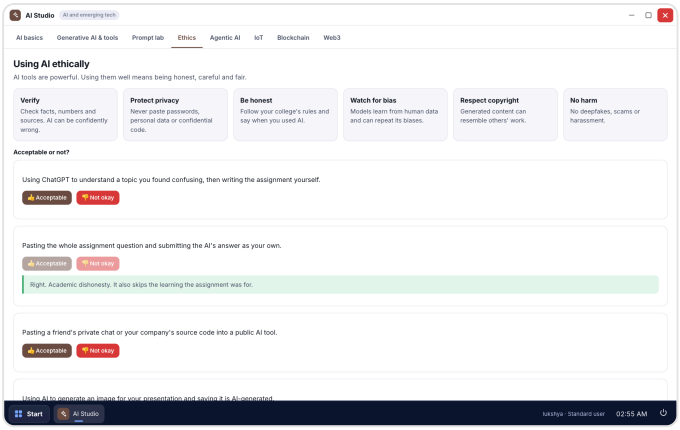


Fig. 62 · **Ethical use of AI.** Principles and "acceptable or not?" scenarios.

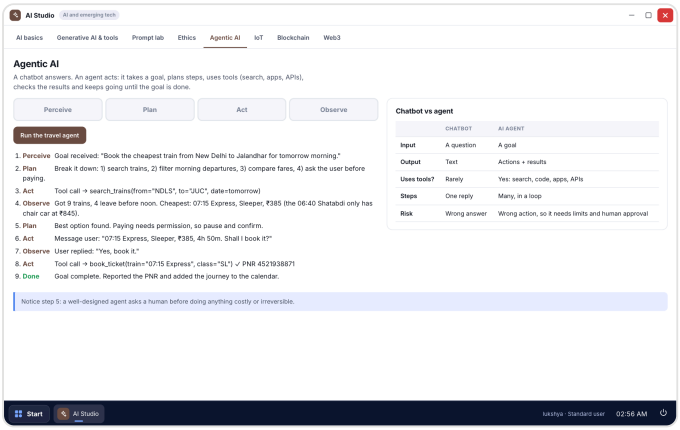


Fig. 63 · **Agentic AI.** An agent books a train: perceive → plan → act (tool calls) → observe, and it asks the user before paying.

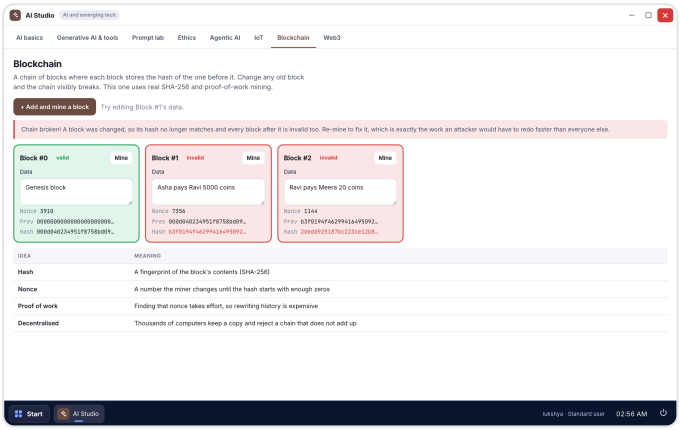


Fig. 64 · **Blockchain.** Real SHA-256 proof-of-work. Changing block 1's data breaks block 1 and every block after it, until they are mined again.

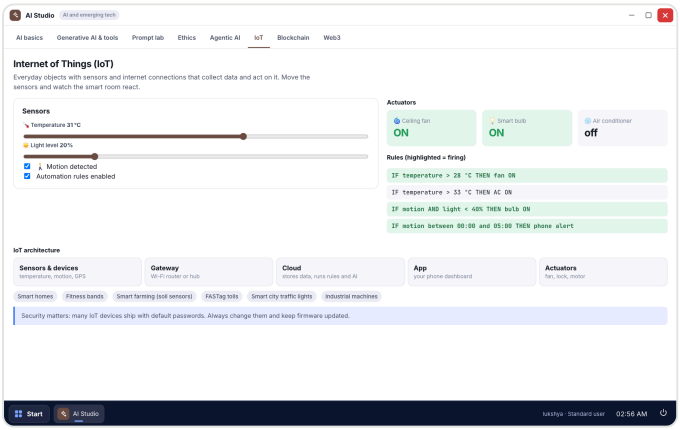


Fig. 65 · **IoT smart room.** Sensors drive rules that switch the fan, AC and lights. Architecture from device to cloud to app.

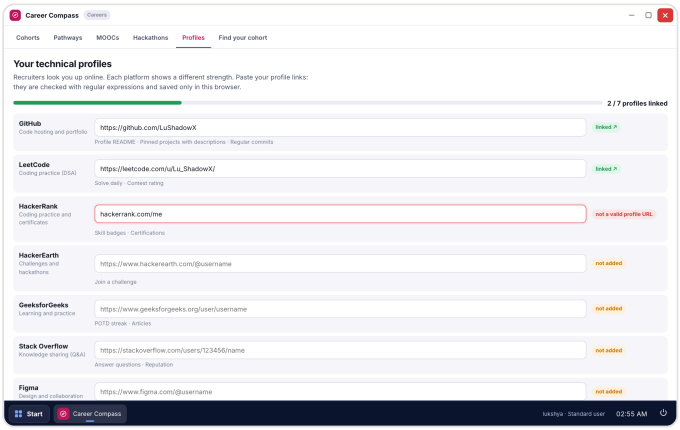


Fig. 66 · **Profile creation.** GitHub, LeetCode, HackerRank, HackerEarth, GeeksforGeeks, Stack Overflow and Figma links are checked with regular expressions (the invalid HackerRank link is flagged).

Flowchart Studio

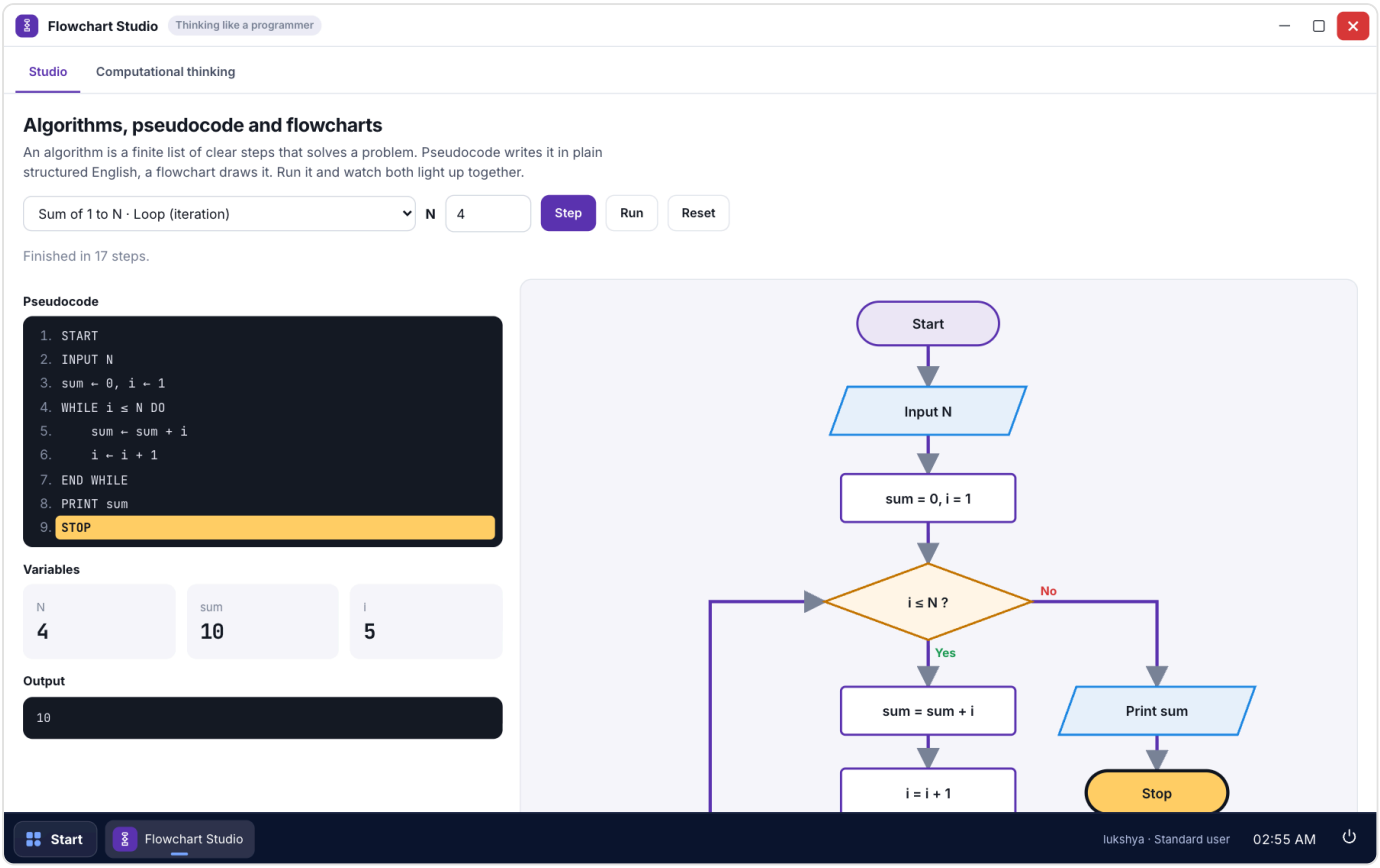


Fig. 67 · **Sum of 1 to N, N = 4.** The pseudocode line, the flowchart symbol and the path taken light up together. The loop back-edge is used four times, and the run ends with sum = 10 after 17 steps (full trace table below the chart).

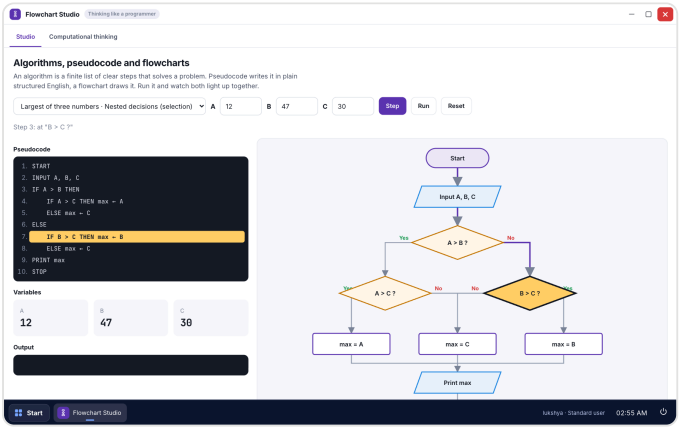


Fig. 68 · **Largest of three**, paused at a nested decision: A = 12, B = 47, C = 30.

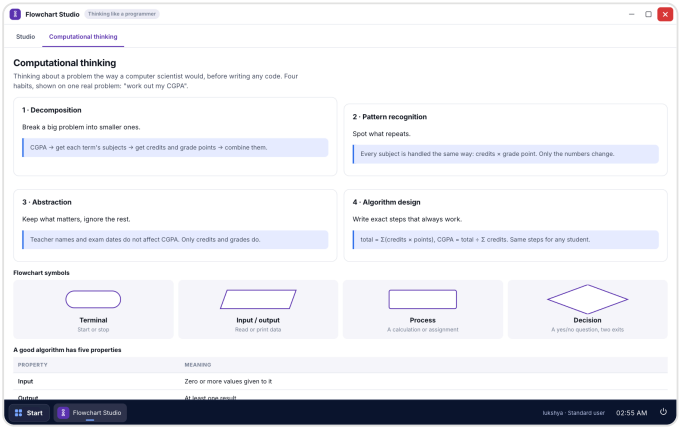


Fig. 69 · **Computational thinking:** decomposition, pattern recognition, abstraction and algorithm design, flowchart symbols, and the properties of a good algorithm.

Five algorithms cover sequence, selection (simple, nested, if–else ladder) and iteration: largest of three, even or odd, sum 1..N, factorial and grade from marks.

Orientation to Computing-I: every topic

The six units of the syllabus I studied. Each topic is listed with where it lives in ByteOS and the figure that shows it.

TOPIC	WHERE IN BYTEOS / WHAT YOU DO	PROOF
Unit 1 · Computer System		
Basic structure of a computer and its components	Inside the Computer › Block diagram: clickable units and a fetch–decode–execute cycle	Fig 7
Memories and their types (RAM, ROM)	Inside the Computer › Memory: hierarchy, SRAM/DRAM, ROM/PROM/EPROM/EEPROM	Fig 9
Secondary storage devices, SSD vs HDD	Inside the Computer › Storage: race, comparison, magnetic/optical/solid state	Fig 10
Processors and GPU	Inside the Computer › CPU vs GPU	Fig 11
PC connection interfaces	Inside the Computer › Ports: USB, Thunderbolt, HDMI, DP, SATA, PCIe, Wi-Fi, Bluetooth, NFC, IR	Fig 12
RAID and RAID levels	Inside the Computer › RAID lab: 0, 1, 5, 6, 10 with failures	Figs 13, 14
Unit 2 · Operating System		
Operating system and its architecture	System Monitor › Architecture: layers and system calls	Fig 20
Kernel and types of kernels	System Monitor › kernel diagrams: monolithic, micro, hybrid, exo	Fig 20
The bootloader	The real ByteOS boot (GRUB menu) + System Monitor › Boot process	Figs 2, 22
Functions of an operating system	System Monitor › Functions + CPU scheduling simulator, Processes tab	Figs 19, 23
Types of operating systems	System Monitor › Types of OS + scenario quiz	Fig 21
Directory hierarchy	Files explorer with descriptions, `tree`, `man hier`	Figs 32, 31
Computer languages	Compiler Lab › Languages: machine, assembly, high-level	Fig 28
Language translators	Compiler Lab: compiler vs interpreter on the same program; assembler explained	Figs 26, 27
Steps in development of a program	Compiler Lab › Program development	Fig 29
Compilation and execution process	Compiler Lab: tokens → AST → assembly → machine code → VM	Figs 24, 25
Unit 3 · Linux Operating System		
Linux overview, architecture and features	Virtual Machines › Linux vs Windows	Fig 35
Linux distributions	Virtual Machines › Distros	Fig 36
Shell commands	Terminal: 45 commands, pipes, redirection	Figs 30, 31
Permissions (chmod)	Terminal + Files: enforced rwx, octal and symbolic chmod, sudo, su	Figs 30, 32

TOPIC	WHERE IN BYTEOS / WHAT YOU DO	PROOF
Windows vs Linux	Virtual Machines › comparison table	Fig 35
Virtual machines	Virtual Machines › Hypervisor: create and run VMs	Fig 34
File system basics and types	Files › File system types, `df -T`	Fig 33
Unit 4 · Cohorts and Skill Sets		
Cohorts: skills, tools, job roles (all 8)	Career Compass › Cohorts	Fig 38
Pathways and MAANG	Career Compass › Pathways	Fig 39
MOOCs	Career Compass › MOOCs	Fig 40
Hackathons and competitions	Career Compass › Hackathons	Fig 41
Unit 5 · Computer Network and Communication		
Networks and types (PAN, LAN, MAN, WAN)	Network Lab › Network types	Fig 48
Network topologies	Network Lab › Topologies with failures	Figs 43 to 45
Network devices	Network Lab › Devices: hub, switch, router, modem, AP, NIC	Fig 46
Client–server model	Network Lab › Client–server	Fig 47
Basic security threats, CIA triad	Security Center › CIA triad	Fig 49
Malware and its types	Security Center › Malware	Fig 54
Phishing (spear, whaling, vishing)	Security Center › Phishing	Fig 53
Password cracking (brute force, dictionary, rainbow)	Security Center › Passwords and Hashing	Figs 50, 51
Multi-factor authentication	Security Center › 2FA with real TOTP codes	Fig 52
User account types and privileges	Security Center › Accounts + Terminal (su, sudo, guest)	Figs 55, 30
Firewall basics	Security Center › Firewall rule tester	Fig 56
Unit 6 · Version Control		
Overview of Git and GitHub	Git Lab: concepts, Git vs GitHub, cheat sheet	Fig 58
Install and configure git, create a GitHub account	Terminal: `git config user.name`; this project itself is on GitHub	§14
Create a local repo, add a file, commit	Terminal + Git Lab missions 1–4	Figs 57, 58
Create a new branch	Terminal `git switch -c`, merge, graph in Git Lab	Fig 58
Introduction to AI and real-life applications	AI Studio › AI basics	Fig 59
Generative AI, types and tools (text, image, video, research)	AI Studio › Generative AI & tools	Fig 60

TOPIC	WHERE IN BYTEOS / WHAT YOU DO	PROOF
Prompt engineering: good vs poor prompts	AI Studio › Prompt lab	Fig 61
Ethical use of AI	AI Studio › Ethics	Fig 62
Profile creation (Figma, GitHub, SO, HackerRank, HackerEarth, GfG, LeetCode)	Career Compass › Profiles	Fig 66

Current CSE111 (session 2026-27): course outcomes

The current version of CSE111 adds number systems, computational thinking and emerging technologies. ByteOS covers all six of its course outcomes too.

CO1
Explain computer systems: organisation, hardware, memory hierarchy, storage, computer languages and number systems

Inside the Computer, Compiler Lab, Number Systems · Figs 7, 9, 15

CO2
Describe operating systems, Linux, networking, virtualization, cloud computing and open-source software

System Monitor, Terminal, Network Lab, Virtual Machines · Figs 20, 30, 34, 37

CO3
Discuss technology cohorts, career pathways, skill resources, MOOCs and hackathons

Career Compass · Figs 38 to 41

CO4
Apply computational thinking with algorithms, pseudocode and flowcharts

Flowchart Studio · Figs 67, 69

CO5
Execute Git and GitHub operations, maintain professional profiles, digital security and safe computing

Terminal git, Git Lab, Career Compass profiles, Security Center · Figs 57, 66, 52

CO6
Understand AI, ML, Generative AI, Agentic AI, IoT, Blockchain and Web3

AI Studio · Figs 59, 63, 65, 64

CURRENT SYLLABUS TOPIC	WHERE IN BYTEOS	PROOF
Computer languages, translators, program development cycle, compilation and execution	Compiler Lab (all tabs)	Figs 24 to 29
Evolution and characteristics of computers, functional units, hardware and software, types of computers	Inside the Computer › Computer basics and Block diagram	Figs 8, 7
CPU and GPU, input and output devices, memory devices	Inside the Computer › CPU vs GPU, basics, Memory	Figs 11, 9
Number systems: binary, decimal, octal, hexadecimal, conversions	Number Systems	Figs 15 to 18
Version control: Git, GitHub, local repo, add, commit, branch	Terminal + Git Lab	Figs 57, 58
Modern AI trends and tools, prompt engineering, ethics	AI Studio	Figs 60 to 62
Profile creation on technical platforms	Career Compass › Profiles	Fig 66

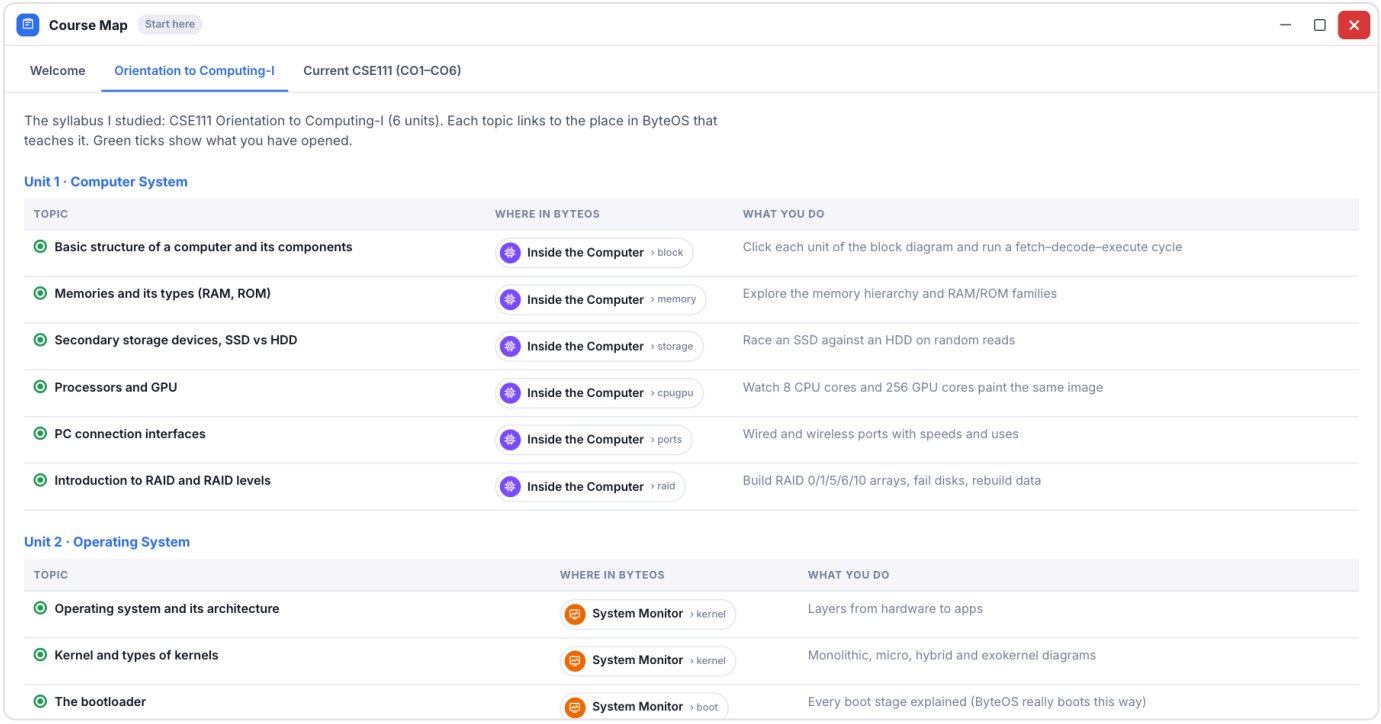


Fig. 70 · **The same mapping inside ByteOS:** the Course Map lists every topic with a link to the app and tab, and ticks the ones you have opened.

How the important parts work

Short extracts from the real source, with what each does and which syllabus idea it implements.

1. Permissions (Linux, Unit 3)

vfs.js · lines 125-130

```
export function can(user, node, action) {
    if (user === 'root') return true;
    const shift = node.owner === user ? 6 : 0;
    const bit = { r: 4, w: 2, x: 1 }[action];
    return Boolean((node.mode >> shift) & bit);
}
```

Each file stores its owner and a mode like `0o644`. Root is always allowed. Otherwise the owner gets the first rwx triple and everyone else the last one, and the bit for r (4), w (2) or x (1) is tested.

vfs.js · lines 146-157

```
function walk(path, user) {
    let node = root;
    let seen = '';
    for (const part of path.split('/').filter(Boolean))
{
        need(user, node, 'x', seen || '/');
        if (node.type !== 'dir') throw new
FsError(`${seen}: Not a directory`);
        node = node.children[part];
        seen += '/' + part;
        if (!node) throw new FsError(`${seen}: No such
file or directory`);
    }
    return node;
}
```

Like Linux, you need execute (search) permission on every directory along a path. That is why a standard user cannot even list `/root`, which has mode 700.

2. Shell parsing (Unit 3)

terminal.js · lines 63-73

```
function tokenize(line) {
    const tokens = [];
    const re = /"([^"]*)"|'([^']*)'|(&&|>>▷|\||
[^\s▷&]+)/g;
    let match;
    while ((match = re.exec(line))) {
        if (match[1] !== undefined) tokens.push({ value:
match[1], quoted: 'double' });
        else if (match[2] !== undefined) tokens.push({
value: match[2], quoted: 'single' });
        else tokens.push({ value: match[3], op: ['&&',
'>>', '>', '|'].includes(match[3]) });
    }
    return tokens;
}
```

A regular expression splits the line into words, quoted strings and the operators `&&`, `|`, `>` and `>>`. The shell then runs `&&` chains, feeds each pipe stage's output into the next, and writes redirections to files.

terminal.js · lines 497-507

```
if (name === 'sudo') {
    if (!USERS[state.user].sudo) throw new
ShellError(`${state.user} is not in the sudoers file.
This incident will be reported.`);
    if (!rest.length) throw new ShellError('usage: sudo
command');
    const before = state.user;
    state.user = 'root';
    try {
        return await runSimple(rest, stdin, 'root');
    } finally {
        state.user = before;
    }
}
```

`sudo` checks if the user is in the sudo group, runs ONE command as root, and always switches back in `finally`, even if the command fails. Guests get the classic "not in the sudoers file" message.

3. Git commits and merges (Unit 6)

git.js · lines 197–211

```
case 'commit': {
    const at = rest.indexOf('-m');
    const message = at === -1 ? '' : rest.slice(at +
1).join(' ');
    if (!message) return ['error: please give a message
with -m "what you changed"'];
    if (!diffTrees(head, repo.index).length) return [`On
branch ${repo.head}`, 'nothing to commit, working tree
clean'];
    const parent = repo.branches[repo.head];
    const { name, email } = identity();
    const time = Date.now();
    const id =
hash(`${parent}${message}${time}${JSON.stringify(repo.index
+ hash(String(time));
    repo.commits[id] = { id, message, parents: parent ?
[parent] : [], tree: { ...repo.index }, author: `${name}
<${email}>`, time, branch: repo.head };
    repo.branches[repo.head] = id;
    save();
    const files = diffTrees(head, repo.index);
    return [`[${repo.head}${parent ? '' : ' (root-
commit)'} ${id.slice(0, 7)}] ${message}`, `
${files.length} file${files.length === 1 ? '' : 's'}
changed`];
}
```

A commit snapshots the staging area (index), records its parent and author, and moves the current branch to point at it. Nothing to commit = no change between HEAD and the index.

git.js · lines 316–322

```
for (const path of new Set([...Object.keys(baseTree),
...Object.keys(oursTree), ...Object.keys(theirsTree)]))
{
    const [b, o, t] = [baseTree[path], oursTree[path],
theirsTree[path]];
    if (o === t) { if (o !== undefined) merged[path] =
o; }
    else if (o === b) { if (t !== undefined)
merged[path] = t; }
    else if (t === b) { if (o !== undefined)
merged[path] = o; }
    else conflicts.push(path);
}
```

Three-way merge: for each file, compare the common ancestor (base), our branch and their branch. If only one side changed it, take that side. If both changed it differently, it is a conflict, and the merge is aborted so nothing is lost.

4. RAID parity (Unit 1)

hardware.js · lines 408-420

```
// GF(2^8) multiply, used for RAID 6's Q parity
function gfMul(a, b) {
    let p = 0;
    for (let i = 0; i < 8; i++) {
        if (b & 1) p ^= a;
        const carry = a & 0x80;
        a = (a << 1) & 0xff;
        if (carry) a ^= 0x1d;
        b >>= 1;
    }
    return p;
}
const gfPow = (n) => { let r = 1; for (let i = 0; i < n;
i++) r = gfMul(r, 2); return r; };
```

RAID 5's P parity is a plain XOR of the data blocks. RAID 6's second parity, Q, multiplies each block by a different power of 2 in the Galois field GF(2^8). That is how two lost blocks can be solved.

hardware.js · lines 453-461

```
function survives(level, n, failed) {
    const f = failed.size;
    if (level === 0) return f === 0;
    if (level === 1) return f < n;
    if (level === 5) return f <= 1;
    if (level === 6) return f <= 2;
    if (level === 10) { for (let p = 0; p < n / 2; p++)
if (failed.has(p * 2) && failed.has(p * 2 + 1)) return
false; return true; }
    return false;
}
```

Fault tolerance per level: RAID 0 survives nothing, RAID 5 one disk, RAID 6 two, and RAID 10 survives as long as no mirror pair loses both disks.

5. Compiler: recursive-descent parser (Unit 2)

compiler.js · lines 62–80

```
const factor = () => {
    const t = peek();
    if (t.type === 'NUMBER') { next(); return { type:
'Number', value: Number(t.value) }; }
    if (t.type === 'IDENT') { next(); return { type:
'Var', name: t.value, line: t.line, col: t.col }; }
    if (t.type === 'LPAREN') { next(); const e = expr();
expect('RPAREN', ')', '")"'); return e; }
    if (t.type === 'OPERATOR' && t.value === '-') {
next(); return { type: 'Negate', value: factor() }; }
    throw new CompileError(`Expected a number, variable
or "(" but found ${t.type === 'EOF' ? 'end of program' :
`"${t.value}"`}`, t.line, t.col);
};
const term = () => {
    let left = factor();
    while (peek().type === 'OPERATOR' &&
'*/'.includes(peek().value)) left = { type: 'Binary',
op: next().value, left, right: factor() };
    return left;
};
const expr = () => {
    let left = term();
    while (peek().type === 'OPERATOR' && '+-
'.includes(peek().value)) left = { type: 'Binary', op:
next().value, left, right: term() };
    return left;
};
const statement = () => {
```

Operator precedence comes from the grammar itself. `expr` handles + and −, and calls `term`. `term` handles × and ÷, and calls `factor`. So in `2 + 3 * 4` the multiplication is grouped first. Each function returns a node of the syntax tree.

6. Round Robin scheduling (Unit 2)

monitor.js · lines 235–256

```
if (algo === 'rr') {
    const queue = [];
    const arrived = new Set();
    const admit = () => list.filter(p => p.arrival <=
time && !arrived.has(p.id)).sort((a, b) => a.arrival -
b.arrival).forEach(p => { arrived.add(p.id);
queue.push(p); });
    admit();
    while (Object.keys(done).length < list.length) {
        if (!queue.length) {
            const next = Math.min(...list.filter(p =>
!arrived.has(p.id)).map(p => p.arrival));
            push('idle', time, next);
            time = next;
            admit();
            continue;
        }
        const p = queue.shift();
        const run = Math.min(quantum, p.left);
        push(p.id, time, time + run);
        time += run;
        p.left -= run;
        admit();
        if (p.left > 0) queue.push(p); else done[p.id] =
time;
    }
} else {
```

A ready queue: each process runs for at most one time quantum, newly arrived processes join the queue, and an unfinished process goes to the back. The tested example matches the textbook result (average waiting time 7.25).

7. Number conversion (current Unit I)

numbers.js · lines 6-12

```
const BASES = { 2: 'Binary', 8: 'Octal', 10: 'Decimal',
16: 'Hexadecimal' };
const VALID = { 2: /^[01]+$/, 8: /^[0-7]+$/, 10:
/^\d+$/, 16: /^[\da-f]+$/i };
const DIGITS = '0123456789ABCDEF';

const parse = (text, base) =>
[...text.toUpperCase()].reduce((acc, d) => acc *
BigInt(base) + BigInt(DIGITS.indexOf(d)), 0n);
const format = (value, base) =>
value.toString(base).toUpperCase();
const group = (bin, size) =>
bin.padStart(Math.ceil(bin.length / size) * size,
'0').match(new RegExp(`.{${size}}`, 'g'));
```

Any base to decimal is positional expansion (multiply the running total by the base and add the digit). `BigInt` keeps it exact for long numbers. The step-by-step tables use the same arithmetic.

8. Network reachability (Unit 5)

network.js · lines 88–106

```
function reachable(graph, from, failedNodes, cutEdges) {
    if (failedNodes.has(from)) return new Set();
    const seen = new Set([from]);
    const prev = new Map();
    const queue = [from];
    while (queue.length) {
        const cur = queue.shift();
        for (const e of graph.edges) {
            if (cutEdges.has(e.id)) continue;
            const other = e.a === cur ? e.b : e.b ===
cur ? e.a : null;
            if (!other || seen.has(other) ||
failedNodes.has(other)) continue;
            seen.add(other);
            prev.set(other, { node: cur, edge: e.id });
            queue.push(other);
        }
    }
    seen.prev = prev;
    return seen;
}
```

Breadth-first search over cables that are not cut and devices that have not failed. It also records the previous hop, so a packet's route can be drawn. The "survives one cut" check runs this once per cable.

9. Two-factor codes, RFC 6238 (Unit 5)

security.js · lines 190–201

```
export async function totp(secretBytes, time =
Date.now(), step = 30, digits = 6) {
    const counter = Math.floor(time / 1000 / step);
    const msg = new ArrayBuffer(8);
    const view = new DataView(msg);
    view.setUint32(0, Math.floor(counter / 2 ** 32));
    view.setUint32(4, counter >>> 0);
    const key = await crypto.subtle.importKey('raw',
secretBytes, { name: 'HMAC', hash: 'SHA-1' }, false,
['sign']);
    const sig = new Uint8Array(await
crypto.subtle.sign('HMAC', key, msg));
    const offset = sig[sig.length - 1] & 0x0f;
    const code = (((sig[offset] & 0x7f) << 24) |
(sig[offset + 1] << 16) | (sig[offset + 2] << 8) |
sig[offset + 3]) % 10 ** digits;
    return String(code).padStart(digits, '0');
}
```

TOTP = HMAC-SHA1(shared secret, number of 30-second steps since 1970), then "dynamic truncation" to 6 digits. It uses the browser's Web Crypto API. It was checked against the official RFC test vectors (94287082 at t = 59 s).

10. Proof-of-work mining (CO6)

ai.js · lines 218-228

```
const mine = async (i) ⇒ {
    const prev = i ? chain[i - 1].hash : '0'.repeat(64);
    const b = chain[i];
    b.nonce = 0;
    while (true) {
        b.hash = await blockHash(b, i, prev);
        if (b.hash.startsWith(DIFFICULTY)) return;
        b.nonce++;
        if (b.nonce % 400 ⩵⩵ 0) { status.textContent =
`Mining block ${i}… tried ${b.nonce} nonces`; await
sleep(0); }
    }
};
```

Try nonce 0, 1, 2… until SHA-256 of (index, data, previous hash, nonce) starts with "000". On average that takes about 4,096 tries, which is what makes rewriting history expensive.

What was checked

Every check below was actually run in a browser (Chrome) or in Node.js, and every result shown is the real output.

#	TEST	RESULT	
1	Open all 14 apps and all 58 tabs	Every screen rendered, zero JavaScript errors in the console	**Pass**
2	Boot: POST → GRUB (Enter) → kernel → login	Reaches the login screen. Esc skips at any stage.	**Pass**
3	`cat /etc/shadow` as lukshya, then with sudo	"Permission denied", then the file contents	**Pass**
4	`su guest` then `sudo ls /root`	"guest is not in the sudoers file"	**Pass**
5	`chmod 600 welcome.txt`	Mode shown as `-rw-------`	**Pass**
6	Pipe and redirect: `cat … | grep -i ram`, `echo … >> file`	Matching line found. File grows to 5 lines.	**Pass**
7	Git: init, add, commit, branch, commit, switch, commit, merge	Three-way merge commit. Log shows 4 commits. Git Lab ticks missions 1–7.	**Pass**
8	RAID 5 with one failed disk / RAID 0 with one failed disk	"Degraded but alive", data rebuilt / "Array failed", data lost	**Pass**
9	Compile the rectangle program	Output 60 and 34 (12 × 5 and (12 + 5) × 2)	**Pass**
10	Program with a missing ";" on line 4	Compiler: error at line 4, nothing runs. Interpreter: prints 82, then stops at line 4.	**Pass**
11	Scheduling P1–P4 (arrival 0,1,2,3 · burst 5,3,8,2), q = 2	Average waiting: FCFS 5.75, SJF 4.00, RR 7.25 (match hand calculation)	**Pass**
12	Conversions	$2026_{10} = 7EA_{16}$, $1101011_{2} = 107_{10} = 153_{8}$, 11000001 signed = −63	**Pass**
13	Ring with one, then two cuts; star with failed switch	6/6, then 3/6 reachable; 1/6	**Pass**
14	Switch: A → C, then C → A	Floods first, learns both MACs, then sends to one port only	**Pass**
15	TOTP against RFC 6238 test vectors	94287082 (t = 59) and 07081804 (t = 1111111109) exactly	**Pass**
16	2FA login with the current code	"Logged in!" Password and code both accepted.	**Pass**
17	Passwords: "password", "P@ssw0rd123", a 4-word passphrase	Dictionary (4 bits), dictionary with rules (14 bits), 171 bits	**Pass**
18	Rainbow table with and without salt	"iloveyou" cracked instantly / not found with a salt	**Pass**
19	Flowchart "sum of 1 to N", N = 4	Output 10 after 17 steps	**Pass**
20	Blockchain: edit block 1, then mine again	Blocks 1 and 2 invalid, then all valid after re-mining	**Pass**
21	Phone width (390 px)	Icons in a grid, windows full screen, no sideways scrolling	**Pass**

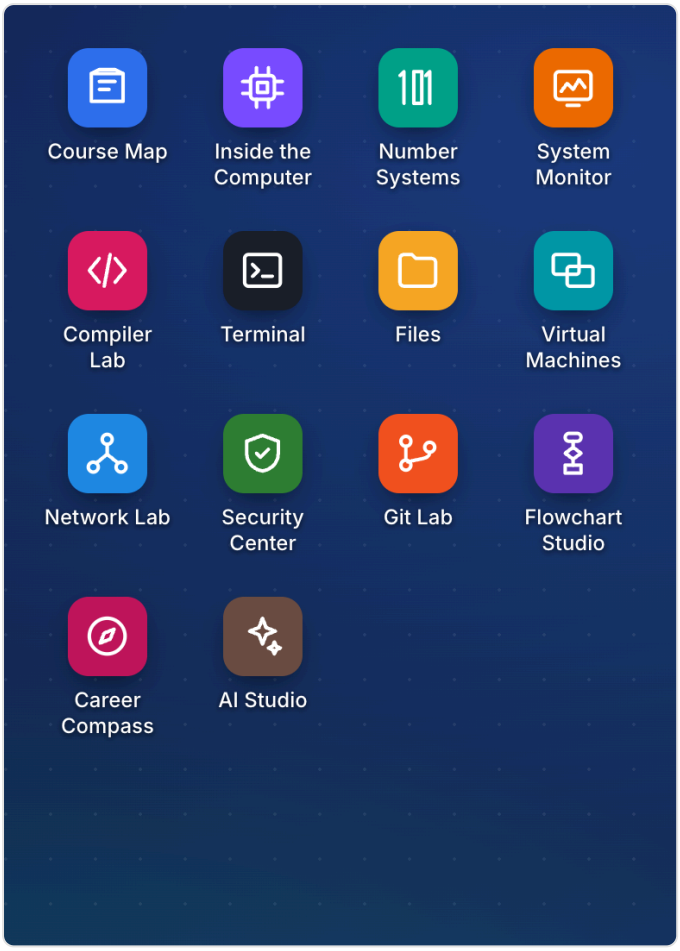


Fig. 71 · **Phone:** the desktop adapts to a grid of icons.

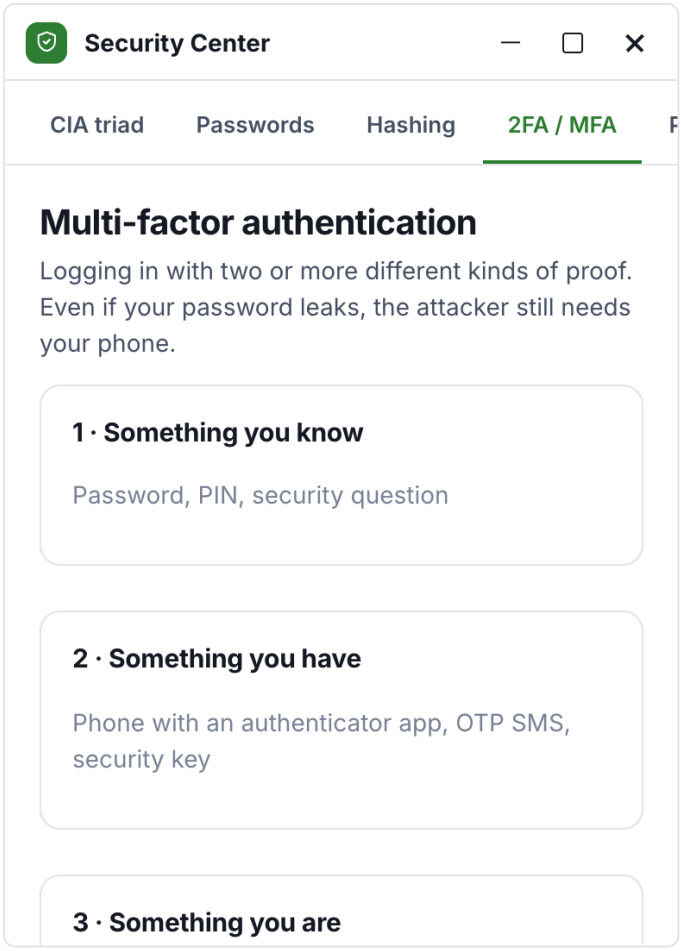


Fig. 72 · **Phone:** apps open full screen.

Hosted on GitHub Pages

The source code is in a public GitHub repository and is served as a website by GitHub Pages from the `main` branch. This also practises the Unit 6 topics for real: the project was built as a git repository with commits, and published on GitHub.

```
git init
git add .
git commit -m "ByteOS: a computer in the browser"
git remote add origin https://github.com/LuShadowX/ByteOS.git
git push -u origin main
# Settings → Pages → Deploy from branch: main / (root)
```

To run it locally: `python3 -m http.server 8000` in the folder, then open `http://localhost:8000`.

What I learned

- **The OS is a set of rules, not a picture.** Making permissions actually block things showed me why `x` on a directory matters and why root is separated from normal users.
- **Redundancy has a price.** The RAID lab made the trade-off clear: RAID 0 is 100% efficient and fragile, RAID 1 is safe and 50% efficient, and parity sits in between.
- **A compiler is just careful steps.** Tokens, a tree, then instructions. The compiler vs interpreter difference became obvious when the interpreter printed output before hitting the error.
- **Security is mostly about people and maths.** Long passphrases beat clever symbols, salts defeat rainbow tables, and 2FA works because a code depends on both the secret and the time.
- **Git is a graph.** Seeing branches and merges drawn live made commits, parents and HEAD click.

> **Links**
> Live: https://lushadowx.github.io/ByteOS/
> Code: https://github.com/LuShadowX/ByteOS